

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Abdurrahman Gilang Harjuna

NIM. 2310817110004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Abdurrahman Gilang Harjuna

NIM : 2310817110004

Menyetujui,

Asisten Praktikum

Mengetahui,

Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar

NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I

NIP. 19881027 2019032013

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR	5
DAFTAR TABEL	6
MODUL 1 : Android Basic With Kotlin.....	7
SOAL 1	7
A. Source Code.....	2
B. Output Program	5
C. Pembahasan	5
MODUL 2 : Android Layout	7
SOAL 1	7
A. Source Code.....	8
B. Output Program	12
C. Pembahasan	12
MODUL 3 : Build a Scrollable List	14
SOAL 1	14
A. Source Code.....	16
B. Output Program	25
C. Pembahasan	25
MODUL 4 : ViewModel & Debugging	30
SOAL 1	30
A. Source Code.....	30
B. Output Program	32
C. Pembahasan	34
MODUL 5 : Connect to the Internet	36
SOAL 1	36
A. Source Code.....	36
B. Output Program	55
C. Pembahasan	56

Tautan Git.....	63
-----------------	----

DAFTAR GAMBAR

MODUL 1

Gambar 1. Screenshot hasil soal 1	5
---	---

MODUL 2

Gambar 2. Screenshot hasil soal 1	12
---	----

MODUL 3

Gambar 3. Screenshot hasil soal 1	25
---	----

MODUL 4

Gambar 4 Screenshot hasil soal 1	32
--	----

Gambar 5 Screenshot hasil soal 1	33
--	----

Gambar 6 Screenshot hasil soal 1	33
--	----

Gambar 7 Screenshot hasil soal 1	34
--	----

MODUL 5

Gambar 8. Screenshot hasil soal 1	55
---	----

Gambar 9. Screenshot hasil soal 1	56
---	----

DAFTAR TABEL

MODUL 1

Table 1. Source Code Jawaban Soal 1	3
Table 2. Source Code Jawaban Soal 1	4

MODUL 2

Table 1. Source Code Jawaban Soal 1	9
Table 2. Source Code Jawaban Soal 1	11

MODUL 3

Table 1. Source Code Jawaban Soal 1	16
Table 2. Source Code Jawaban Soal 1	17
Table 3. Source Code Jawaban Soal 1	19
Table 4. Source Code Jawaban Soal 1	19
Table 5. Source Code Jawaban Soal 1	21
Table 6. Source Code Jawaban Soal 1	22
Table 7. Source Code Jawaban Soal 11	23
Table 8. Source Code Jawaban Soal 1	24

MODUL 4

Table 1. Source Code Jawaban Soal 1	31
Table 2. Source Code Jawaban Soal 1	31
Table 3. Source Code Jawaban Soal 1	32

MODUL 5

Table 1. Source Code Jawaban Soal 1	38
Table 2. Source Code Jawaban Soal 1	41
Table 3. Source Code Jawaban Soal 1	43
Table 4. Source Code Jawaban Soal 1	44
Table 5. Source Code Jawaban Soal 1	44
Table 6. Source Code Jawaban Soal 1	45
Table 7. Source Code Jawaban Soal 1	45
Table 8. Source Code Jawaban Soal 1	48
Table 9. Source Code Jawaban Soal 1	48
Table 10. Source Code Jawaban Soal 1	49
Table 11. Source Code Jawaban Soal 1	49
Table 12. Source Code Jawaban Soal 1	53
Table 13. Source Code Jawaban Soal 1	55

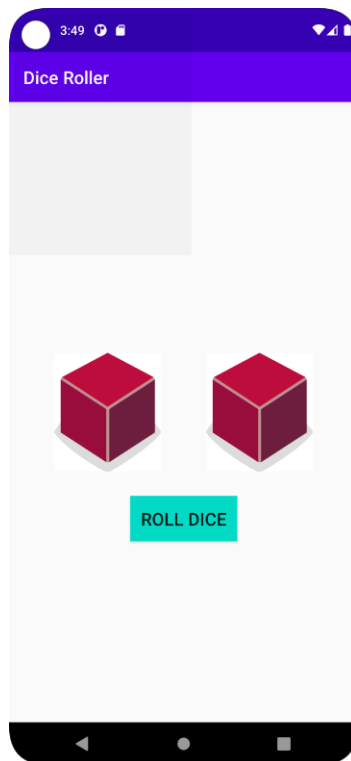
MODUL 1 : Android Basic With Kotlin

SOAL 1

Soal Praktikum:

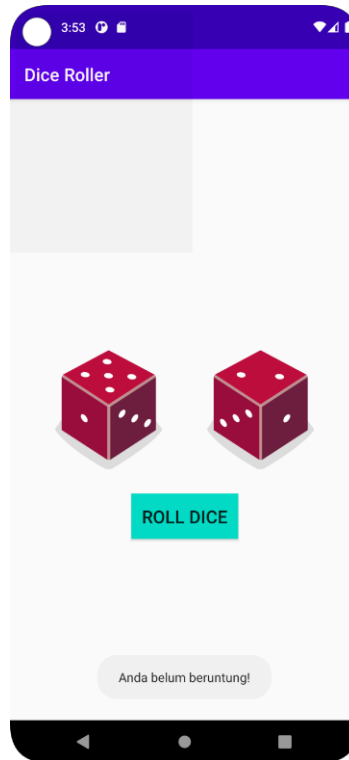
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



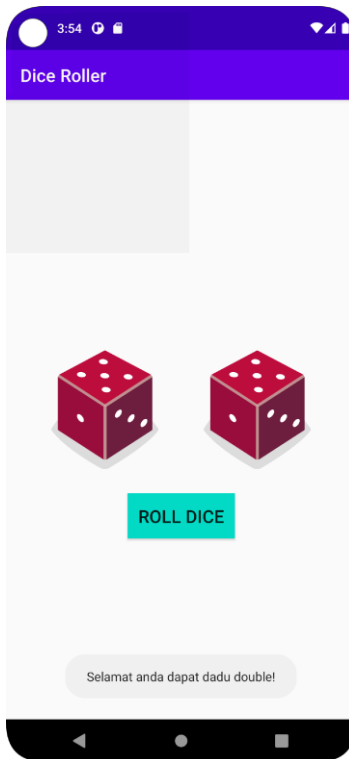
Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3 Tampilan Roll Dadu Double

A. Source Code

1. MainActivity.kt

```
1 package com.example.diceroller
2
3 import android.os.Bundle
4 import android.widget.Button
5 import androidx.appcompat.app.AppCompatActivity
6 import android.widget.ImageView
7 import android.widget.Toast
8
9 class MainActivity : AppCompatActivity() {
10     override fun onCreate(savedInstanceState: Bundle?) {
11         super.onCreate(savedInstanceState)
12         setContentView(R.layout.activity_main)
13
14         val diceImage01: ImageView = findViewById(R.id.Dice1)
15         diceImage01.setImageResource(R.drawable.dice_0)
16
17         val diceImage02: ImageView = findViewById(R.id.Dice2)
18         diceImage02.setImageResource(R.drawable.dice_0)
19
20         val rollButton: Button = findViewById(R.id.Buttonn)
21         rollButton.setOnClickListener { diceRoller() }
22     }
23 }
```

14	<code>private fun diceRoller() {</code>
15	<code> val dice1 = Dice(6)</code>
16	<code> val diceRoll1 = dice1.roll()</code>
17	<code> val diceImage1: ImageView = findViewById(R.id.Dice1)</code>
18	<code> when (diceRoll1) {</code>
19	<code> 1 -> diceImage1.setImageResource(R.drawable.dice_1)</code>
20	<code> 2 -> diceImage1.setImageResource(R.drawable.dice_2)</code>
21	<code> 3 -> diceImage1.setImageResource(R.drawable.dice_3)</code>
	<code> 4 -> diceImage1.setImageResource(R.drawable.dice_4)</code>
	<code> 5 -> diceImage1.setImageResource(R.drawable.dice_5)</code>
	<code> 6 -> diceImage1.setImageResource(R.drawable.dice_6)</code>
	<code> }</code>
	<code> val dice2 = Dice(6)</code>
	<code> val diceRoll2 = dice2.roll()</code>
	<code> val diceImage2: ImageView = findViewById(R.id.Dice2)</code>
	<code> when (diceRoll2) {</code>
	<code> 1 -> diceImage2.setImageResource(R.drawable.dice_1)</code>
	<code> 2 -> diceImage2.setImageResource(R.drawable.dice_2)</code>
	<code> 3 -> diceImage2.setImageResource(R.drawable.dice_3)</code>
	<code> 4 -> diceImage2.setImageResource(R.drawable.dice_4)</code>
	<code> 5 -> diceImage2.setImageResource(R.drawable.dice_5)</code>
	<code> 6 -> diceImage2.setImageResource(R.drawable.dice_6)</code>
	<code> }</code>
	<code> if (diceRoll1 == diceRoll2) {</code>
	<code> val toast = Toast.makeText(this, "Selamat anda dapat dadu</code>
	<code>double!", Toast.LENGTH_SHORT)</code>
	<code> toast.show()</code>
	<code> } else {</code>
	<code> val toast = Toast.makeText(this, "Anda belum beruntung!",</code>
	<code>Toast.LENGTH_SHORT)</code>
	<code> toast.show() }</code>
	<code> }</code>
	<code>}</code>
	<code>class Dice(private val numSides: Int) {</code>
	<code> fun roll(): Int {</code>
	<code> return (1..numSides).random()</code>
	<code> }</code>
	<code>}</code>

Table 1. Source Code Jawaban Soal 1

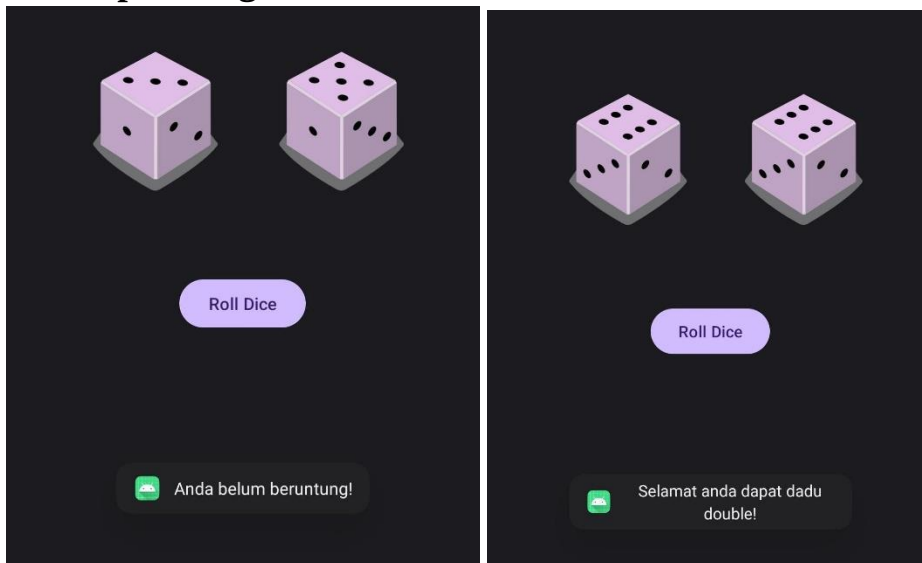
2. activity_main.xml

1	<code><?xml version="1.0" encoding="utf-8"?></code>
2	<code><androidx.constraintlayout.widget.ConstraintLayout</code>
3	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4	<code>xmlns:app="http://schemas.android.com/apk/res-auto"</code>
	<code>xmlns:tools="http://schemas.android.com/tools"</code>
	<code>android:id="@+id/main"</code>
	<code>android:layout_width="match_parent"</code>
	<code>android:layout_height="match_parent"</code>

5	<code>tools:context=".MainActivity"></code>
6	<code><Button</code>
7	<code> android:id="@+id/Buttonn"</code>
8	<code> android:layout_width="wrap_content"</code>
9	<code> android:layout_height="wrap_content"</code>
10	<code> android:layout_margin="52dp"</code>
11	<code> android:text="Roll Dice"</code>
	<code> app:layout_constraintEnd_toEndOf="parent"</code>
	<code> app:layout_constraintStart_toStartOf="parent"</code>
	<code> app:layout_constraintTop_toBottomOf="@+id/Dice1"</code>
	<code> app:layout_constraintHorizontal_bias="0.498"</code>
	<code> tools:ignore="UnknownId" /></code>
	<code><ImageView</code>
	<code> android:id="@+id/Dice1"</code>
	<code> android:layout_width="150dp"</code>
	<code> android:layout_height="150dp"</code>
	<code> app:layout_constraintBottom_toBottomOf="parent"</code>
	<code> app:layout_constraintEnd_toEndOf="parent"</code>
	<code> app:layout_constraintHorizontal_bias="0.201"</code>
	<code> app:layout_constraintStart_toStartOf="parent"</code>
	<code> app:layout_constraintTop_toTopOf="parent"</code>
	<code> app:layout_constraintVertical_bias="0.449"</code>
	<code> app:srcCompat="@drawable/dice_1"</code>
	<code> tools:layout_editor_absoluteX="83dp"</code>
	<code> tools:layout_editor_absoluteY="268dp" /></code>
	<code><ImageView</code>
	<code> android:id="@+id/Dice2"</code>
	<code> android:layout_width="150dp"</code>
	<code> android:layout_height="150dp"</code>
	<code> app:layout_constraintBottom_toBottomOf="parent"</code>
	<code> app:layout_constraintEnd_toEndOf="parent"</code>
	<code> app:layout_constraintHorizontal_bias="0.837"</code>
	<code> app:layout_constraintStart_toStartOf="parent"</code>
	<code> app:layout_constraintTop_toTopOf="parent"</code>
	<code> app:layout_constraintVertical_bias="0.449"</code>
	<code> app:srcCompat="@drawable/dice_2" /></code>
	<code></androidx.constraintlayout.widget.ConstraintLayout></code>

Table 2. Source Code Jawaban Soal 1

B. Output Program



Gambar 1. Screenshot hasil soal 1

C. Pembahasan

1. MainActivity.kt:

MainActivity adalah activity utama dari aplikasi. Turunan dari AppCompatActivity, yang merupakan base class umum untuk activity modern di android. onCreate() dipanggil saat activity pertama dibuat. setContentView(R.layout.activity_main) menghubungkan file XML sebagai tampilan utama aplikasi.

```
val diceImage01: ImageView = findViewById(R.id.Dice1)
diceImage01.setImageResource(R.drawable.dice_0) dan val diceImage02: ImageView =
findViewById(R.id.Dice2) diceImage02.setImageResource(R.drawable.dice_0) kedua kode
ini mengambil dua ImgeView dari layout (dadu 1 dan 2). Gambar diset ke dice_0, artinya
sebelum tombol ditekan, gambar dadu kosong.
```

```
val rollButton: Button = findViewById(R.id.Buttonnn) rollButton.setOnClickListener {
diceRoller() } menghubungkan tombol dari layout dengan kode, saat tombol diklik akan
menjalankan fungsi diceRoller().private fun diceRoller() { ... } val dice1 = Dice(6) val
diceRoll1 = dice1.roll() membuat dice dengan 6 sisi, roll() menghasilkan angka acak antara
1-6.
```

```
val diceImage1: ImageView = findViewById(R.id.Dice1) when (diceRoll1) { 1 ->
diceImage1.setImageResource(R.drawable.dice_1) ...} Berdasarkan hasil roll, gambar dadu
diubah sesuai angka (dice_1.png sampai dice_6.png). val dice2 = Dice(6) val diceRoll2 =
dice2.roll() dan val diceImage2: ImageView = findViewById(R.id.Dice2) when (diceRoll2)
```

{ ... } sama seperti dadu pertama tapi ini untuk dadu kedua.

Lanjut ke toast, menampilkan pesan kecil di layar, disini cek apakah kedua dadu sama atau tidak. Jika sama berarti dapat dadu double, jika beda maka belum beruntung. class Dice(private val numSides: Int) { ini adalah class custom dadu, numSides ini jumlah sisi dadu berupa angka. fun roll(): Int { return (1..numSides).random() } } Fungsi roll() menghasilkan angka acak dari 1 sampai numSides yaitu 6.

2. activity_main.xml

<androidx.constraintlayout.widget.ConstraintLayout> jenis layout yang memungkinkan untuk mengatur setiap elemen dengan menghubungkan ke elemen lain atau ke parent-nya. Ini root layout artinya semua elemen UI di dalamnya akan menggunakan constraint untuk pengaturan posisinya.

ImageView Dice 1 ini adalah gambar dadu pertama. @+id/Dice1 ini Id untuk mengakses dari kode kotlin. layout_width, layout_height di set ke 150dp, ukuran tetap. app:srcCompat="@drawable/dice_1" menampilkan gambar dadu (dice_1.png dari folder res/drawable). Sisa constraint untuk mengatur posisi agar bisa di tengah bagian kiri.

ImageView Dice 2 ini adalah dadu kedua. @+id/Dice2 sebagai id sisanya sama seperti dadu pertama. Untuk posisi diatur dengan constraint agar berada di tengah bagian kanan.

Button roll dice dengan id Buttonn, tombol teks Roll Dice. Diberi margin 52dp sekelilingnya. Posisinya diletakkan dibawah dice 1 karena Top_toBottomOf="@+id/Dice1" dan diatur lagi dengan Start_toStartOf="parent", End_toEndOf="parent" dan Horizontal_bias="0.498" agar bisa berada di tengah antara dice 1 dan 2.

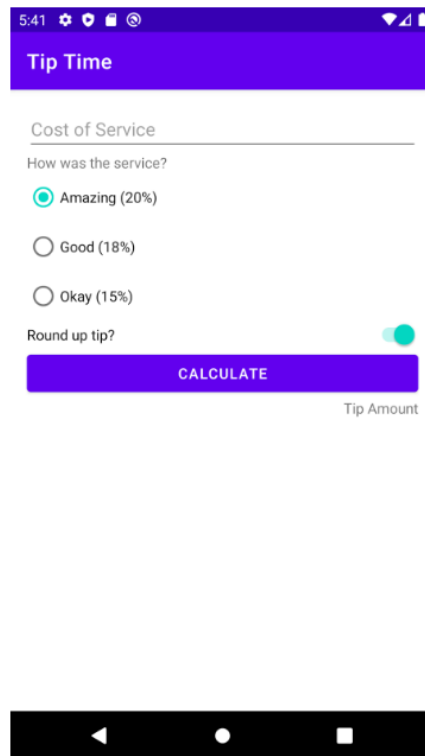
MODUL 2 : Android Layout

SOAL 1

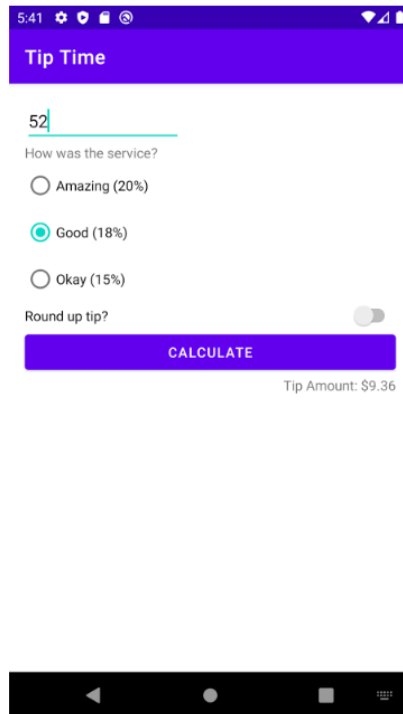
Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1 Tampilan Awal Aplikasi



Gambar 2 Tampilan Aplikasi Setelah Dijalankan

A. Source Code

3. MainActivity.kt

```

1 package com.example.calculatortip
2
3 import android.os.Bundle
4 import android.widget.*
5 import androidx.appcompat.app.AppCompatActivity
6 import androidx.appcompat.widget.SwitchCompat
7 import kotlin.math.ceil
8
9 class MainActivity : AppCompatActivity() {
10     override fun onCreate(savedInstanceState: Bundle?) {
11         super.onCreate(savedInstanceState)
12         setContentView(R.layout.activity_main)
13
14         val costOfService =
15             findViewById<EditText>(R.id.cost_of_service)
16         val tipOptions = findViewById<RadioGroup>(R.id.tip_options)
17         val roundUpSwitch =
18             findViewById<SwitchCompat>(R.id.round_up_switch)
19         val calculateButton =
20             findViewById<Button>(R.id.calculate_button)
21         val tipResult = findViewById<TextView>(R.id.tip_result)
22
23         calculateButton.setOnClickListener {

```

15	<code>val cost = costOfService.text.toString().toDoubleOrNull()</code>
16	<code>if (cost == null) {</code>
17	<code>tipResult.text = ""</code>
18	<code>Toast.makeText(this,</code>
19	<code>getString(R.string.cost_of_service_hint), Toast.LENGTH_SHORT).show()</code>
20	<code>return@setOnClickListener</code>
21	<code>}</code>
	<code>val tipPercentage = when</code>
	<code>(tipOptions.checkedRadioButtonId) {</code>
	<code>R.id.option_twenty_percent -> 0.20</code>
	<code>R.id.option_eighteen_percent -> 0.18</code>
	<code>else -> 0.15</code>
	<code>}</code>
	<code>var tip = cost * tipPercentage</code>
	<code>if (roundUpSwitch.isChecked) {</code>
	<code>tip = ceil(tip)</code>
	<code>}</code>
	<code>val formattedTip = getString(R.string.tip_result) + " \$"</code>
	<code>+ "%.2f".format(tip)</code>
	<code>tipResult.text = formattedTip</code>
	<code>}</code>
	<code>}</code>

Table 3. Source Code Jawaban Soal 1

4. activity_main.xml

1	<code><?xml version="1.0" encoding="utf-8"?></code>
2	<code><LinearLayout</code>
3	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4	<code>xmlns:app="http://schemas.android.com/apk/res-auto"</code>
5	<code>xmlns:tools="http://schemas.android.com/tools"</code>
6	<code>android:layout_width="match_parent"</code>
7	<code>android:layout_height="match_parent"</code>
8	<code>android:orientation="vertical"</code>
9	<code>android:padding="16dp"</code>
10	<code>tools:context=".MainActivity"></code>
11	<code><EditText</code>
	<code>android:id="@+id/cost_of_service"</code>
	<code>android:layout_width="match_parent"</code>
	<code>android:layout_height="wrap_content"</code>
	<code>android:hint="@string/cost_of_service_hint"</code>
	<code>android:inputType="numberDecimal"</code>
	<code>android:autofillHints="postalAddress"</code>
	<code>android:textColor="@android:color/black"</code>
	<code>android:textSize="16sp"</code>
	<code>android:minHeight="48dp"/></code>


```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/how_was_service"
    android:paddingTop="16dp"
    android:textColor="@android:color/black"
    android:textSize="16sp"/>

<RadioGroup
    android:id="@+id/tip_options"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">

    <RadioButton
        android:id="@+id/option_twenty_percent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/option_twenty_percent"/>

    <RadioButton
        android:id="@+id/option_eighteen_percent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/option_eighteen_percent"/>

    <RadioButton
        android:id="@+id/option_fifteen_percent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/option_fifteen_percent"/>
</RadioGroup>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingTop="16dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="@string/round_up_tip"
        android:textColor="@android:color/black"
        android:textSize="16sp"/>

    <androidx.appcompat.widget.SwitchCompat
        android:id="@+id/round_up_switch"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:minHeight="48dp"
        android:minWidth="48dp"/>

```

	<pre> </LinearLayout> <Button android:id="@+id/calculate_button" android:layout_width="match_parent" android:layout_height="wrap_content" android:text="@string/calculate" android:layout_marginTop="16dp" android:minHeight="48dp"/> <TextView android:id="@+id/tip_result" android:layout_width="match_parent" android:layout_height="wrap_content" android:paddingTop="16dp" android:text="@string/tip_result" android:textColor="@android:color/black" android:textSize="16sp" android:gravity="end" android:textAlignment="viewEnd" /> </LinearLayout> </pre>
--	--

Table 4. Source Code Jawaban Soal 1

B. Output Program

The image displays two side-by-side screenshots of a mobile application interface for calculating a tip. Both screenshots show a status bar at the top with the time 21:00, data usage (0,3KB/d on the left, 4,7KB/d on the right), and battery level (55% on the left, 54% on the right). The app has a light purple background.

Left Screenshot:

- Cost of Service:** An input field that is currently empty.
- How was the service?:** Three radio button options:
- Amazing (20%)
- Good (18%)
- Okay (15%)
- Round up tip?:** A toggle switch that is currently turned off.
- CALCULATE:** A purple button.
- Tip Amount:** A label followed by an empty input field.

Right Screenshot:

- Cost of Service:** An input field containing the value "52".
- How was the service?:** Three radio button options:
- Amazing (20%)
- Good (18%) (This option is selected, indicated by a purple dot.)
- Okay (15%)
- Round up tip?:** A toggle switch that is currently turned off.
- CALCULATE:** A purple button.
- Tip Amount:** A label followed by an input field containing the value "\$9,36".

Gambar 2. Screenshot hasil soal 1

C. Pembahasan

1. MainActivity.kt:

Pertama terdapat paket dan Import ini menyatakan bahwa file ini termasuk dalam package `com.example.calculatortip`. Kemudian beberapa library penting diimpor, seperti `Bundle`, `EditText`, `RadioGroup`, `SwitchCompat`, dan `ceil()` dari Kotlin untuk membulatkan angka ke atas.

Selanjutnya membuat `MainActivity` yang merupakan turunan dari `AppCompatActivity`. Di dalam metode `onCreate()`, layout dari `activity_main.xml` diatur agar menjadi tampilan utama saat activity dimulai.

Pada bagian `val` ada lima baris yang mengambil referensi dari komponen UI di XML dengan menggunakan `findViewById()`. Masing-masing disimpan dalam variabel untuk digunakan dalam logika aplikasi. `calculateButton.setOnClickListener {` adalah Event listener yang akan berjalan saat tombol "Calculate" ditekan.

`val cost = costOfService.text.toString().toDoubleOrNull()` Input dari pengguna diambil dan dikonversi menjadi angka (`Double`). Lalu dibawahnya ada kondisional yang mana Jika input tidak valid (misalnya kosong atau bukan angka), maka hasil akan dikosongkan dan pengguna akan mendapat toast message untuk memberi tahu kesalahan. Selanjutnya pada `val tipPercentage`. User diminta memilih radio button untuk menentukan persentase tip. Jika tidak ada yang dipilih maka akan menggunakan 15%.

Selanjutnya tip dihitung dari biaya dikalikan dengan persentasenya. Jika opsi pembulatan diaktifkan maka tip akan dibulatkan. Terakhir hasil tip diformat ke dua angka decimal dan ditampilkan.

2. activity_main.xml

Pertama ada layout utama yaitu `LinearLayout` dengan orientasi vertikal. Ini artinya semua elemen UInya akan disusun dari atas ke bawah. Layout ini mengisi seluruh lebar dan tinggi layar (`match_parent`) dan memberikan padding sebesar 16dp di sekelilingnya. Elemen `tools:context` hanya digunakan oleh Android Studio untuk keperluan preview dan tidak memengaruhi jalannya aplikasi.

Kedua pada bagian dimana pengguna harus inputkan angka, ini menampilkan kolom input (`EditText`) di mana pengguna bisa mengetikkan biaya layanan. `inputType` diatur ke `numberDecimal` agar hanya menerima angka desimal. Teks petunjuk (`hint`) ditarik dari file `strings.xml` dan teks yang diketik ditampilkan dengan warna hitam dan ukuran 16sp.

Setelah kolom input pengguna, pengguna diberikan pertanyaan "Bagaimana kualitas layanan?" lewat `TextView`, diikuti dengan `RadioGroup` yang terdiri dari tiga `RadioButton`—20%, 18%, dan 15%. Hanya satu opsi yang bisa dipilih karena elemen-elemen ini berada dalam `RadioGroup`.

Selanjutnya pada bagian `LinearLayout` horizontal ini yang memuat teks "Bulatkan tip ke atas" dan sebuah switch (`SwitchCompat`). Switch ini dapat diaktifkan atau dinonaktifkan oleh pengguna, untuk menentukan apakah hasil tip akan dibulatkan ke atas.

Lanjut ke tombol calculate ini akan menjalankan proses kalkulasi tip saat ditekan. Lebarnya memenuhi seluruh layar (match_parent) dan diberi jarak atas 16dp untuk pemisahan visual dari elemen sebelumnya.

Bagian akhir, `TextView` ini akan menampilkan hasil dari perhitungan tip. Letaknya di bawah tombol dan teksnya diratakan ke kanan (`gravity="end"`). Nilai awal teks diambil dari `strings.xml`.

MODUL 3 : Build a Scrollable List

SOAL 1

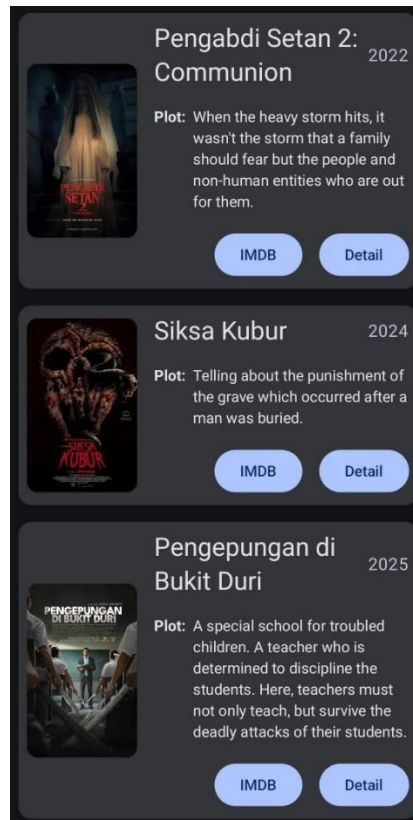
Soal Praktikum:

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Dusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

A. Source Code

1. MainActivity.kt

```
1 package com.example.zennfit
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.zennfit.databinding.ActivityMainBinding
6
7 class MainActivity : AppCompatActivity() {
8
9     data class AlatGym(
10         val nama: String,
11         val descSingkat: String,
12         val deskripsi: String,
13         val gambarResId: Int,
14         val gifResId: Int
15     )
16
17     private lateinit var binding: ActivityMainBinding
18
19     override fun onCreate(savedInstanceState: Bundle?) {
20         super.onCreate(savedInstanceState)
21         binding = ActivityMainBinding.inflate(layoutInflater)
22         setContentView(binding.root)
23
24         // Tampilkan HomeFragment sebagai default
25         if (savedInstanceState == null) {
26             supportFragmentManager.beginTransaction()
27                 .replace(R.id.fragment_container, HomeFragment())
28                 .commit()
29         }
30     }
31 }
```

Table 5. Source Code Jawaban Soal 1

2. activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical">
```

5	<code><TextView</code>
6	<code> android:layout_width="match_parent"</code>
7	<code> android:layout_height="wrap_content"</code>
8	<code> android:gravity="center"</code>
9	<code> android:padding="16dp"</code>
10	<code> android:text="ZennFit"</code>
11	<code> android:background="@color/black"</code>
	<code> android:textColor="@android:color/white"</code>
	<code> android:textSize="24sp" /></code>
	<code><FrameLayout</code>
	<code> android:id="@+id/fragment_container"</code>
	<code> android:layout_width="match_parent"</code>
	<code> android:layout_height="match_parent" /></code>
	<code></LinearLayout></code>

Table 6. Source Code Jawaban Soal 1

3. HomeFragment.kt

1	<code>package com.example.zennfit</code>
2	<code>import android.os.Bundle</code>
3	<code>import android.view.LayoutInflater</code>
4	<code>import android.view.View</code>
5	<code>import android.view.ViewGroup</code>
6	<code>import androidx.fragment.app.Fragment</code>
7	<code>import androidx.recyclerview.widget.LinearLayoutManager</code>
8	<code>import com.example.zennfit.databinding.FragmentHomeBinding</code>
9	<code>class HomeFragment : Fragment() {</code>
10	<code> private var _binding: FragmentHomeBinding? = null</code>
11	<code> private val binding get() = _binding!!</code>
	<code> // Gunakan data class dari MainActivity</code>
	<code> private lateinit var alatList: List<MainActivity.AlatGym></code>
	<code> override fun onCreateView(</code>
	<code> inflater: LayoutInflater, container: ViewGroup?,</code>
	<code> savedInstanceState: Bundle?</code>
	<code>): View {</code>
	<code> _binding = FragmentHomeBinding.inflate(inflater, container,</code>
	<code>false)</code>
	<code> return binding.root</code>
	<code> }</code>
	<code> override fun onViewCreated(view: View, savedInstanceState:</code>
	<code>Bundle?) {</code>
	<code> super.onViewCreated(view, savedInstanceState)</code>
	<code> // Pindahkan pengisian alatList ke onViewCreated, karena</code>


```

sudah ada konteks
    val context = requireContext()
    alatList = listOf(
        MainActivity.AlatGym(
            "BenchPress",
            "Melatih otot dada dengan cara mendorong barbel dari
posisi telentang di bangku datar.",
            context.getString(R.string.desc_benchpress),
            R.drawable.benchpress,
            R.drawable.incline_benchpress
        ),
        MainActivity.AlatGym(
            "Row Cable",
            "Melatih otot punggung dengan gerakan mendayung dari
posisi duduk.",
            context.getString(R.string.desc_row),
            R.drawable.cablerow,
            R.drawable._seated
        ),
        MainActivity.AlatGym(
            "Shoulderpress Machine",
            "Melatih otot bahu dengan mendorong beban ke atas
dari posisi duduk.",
            context.getString(R.string.desc_shoulder),
            R.drawable.shoulderpress,
            R.drawable.shoulderpress_machine
        ),
        MainActivity.AlatGym(
            "Legpress Machine",
            "Melatih otot kaki dengan menekan beban menjauh
menggunakan kedua kaki.",
            context.getString(R.string.desc_legpress),
            R.drawable.legpress,
            R.drawable.legpressmachine
        ),
        MainActivity.AlatGym(
            "Fly Machine",
            "Melatih otot dada dengan gerakan merapatkan lengan
dari samping ke depan seperti memeluk.",
            context.getString(R.string.desc_fly),
            R.drawable.fly,
            R.drawable.fly_machine
        )
    )

    binding.recyclerView.layoutManager =
LinearLayoutManager(requireContext())
    binding.recyclerView.adapter = AlatGymAdapter(alatList) {
alat ->
        // Kirim data ke DetailFragment menggunakan Bundle
        val fragment = DetailFragment().apply {
            arguments = Bundle().apply {
                putString("nama", alat.nama)
            }
        }
    }
}

```

	<pre> putString("deskripsi", alat.deskripsi) putInt("gambar", alat.gambarResId) putInt("gif", alat.gifResId) } } // Navigasi ke DetailFragment parentFragmentManager.beginTransaction() .replace(R.id.fragment_container, fragment) .addToBackStack(null) .commit() } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	---

Table 7. Source Code Jawaban Soal 1

4. Fragment_home.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical">
7	<androidx.recyclerview.widget.RecyclerView
8	android:id="@+id/recyclerView"
	android:layout_width="match_parent"
	android:layout_height="match_parent"
	android:padding="8dp"
	android:contentDescription="Daftar alat gym" />
	</LinearLayout>

Table 8. Source Code Jawaban Soal 1

5. DetailFragment.kt

1	package com.example.zennfit
2	import android.os.Bundle
3	import android.view.LayoutInflater
4	import android.view.View
5	import android.view.ViewGroup
6	import androidx.fragment.app.Fragment

```

5 import com.bumptech.glide.Glide
6 import com.example.zennfit.databinding.FragmentDetailBinding
7
8 class DetailFragment : Fragment() {
9
10     private var _binding: FragmentDetailBinding? = null
11     private val binding get() = _binding!!
12
13     companion object {
14         fun newInstance(nama: String, deskripsi: String, gambar: Int,
15 gif: Int): DetailFragment {
16             val fragment = DetailFragment()
17             val bundle = Bundle().apply {
18                 putString("nama", nama)
19                 putString("deskripsi", deskripsi)
20                 putInt("gambar", gambar)
21                 putInt("gif", gif)
22             }
23             fragment.arguments = bundle
24             return fragment
25         }
26     }
27
28     override fun onCreateView(
29         inflater: LayoutInflater, container: ViewGroup?,
30         savedInstanceState: Bundle?
31     ): View {
32         _binding = FragmentDetailBinding.inflate(inflater, container,
33 false)
34         return binding.root
35     }
36
37     override fun onViewCreated(view: View, savedInstanceState:
38 Bundle?) {
39         super.onViewCreated(view, savedInstanceState)
40
41         val nama = arguments?.getString("nama")
42         val deskripsi = arguments?.getString("deskripsi")
43         val gambar = arguments?.getInt("gambar") ?: 0
44         val gif = arguments?.getInt("gif") ?: 0
45
46         activity?.title = nama
47         binding.namaAlat.text = nama
48         binding.deskripsiText.text = deskripsi
49         binding.alatImage2.setImageResource(gambar)
50         Glide.with(this).asGif().load(gif).into(binding.Gerakan)
51     }
52
53     override fun onDestroyView() {
54         super.onDestroyView()
55         _binding = null
56     }
57 }

```

	<pre> } } </pre>
--	------------------------------

Table 9. Source Code Jawaban Soal 1

6. Fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?> <ScrollView
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
	android:layout_height="match_parent">
4	<LinearLayout
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
	android:orientation="vertical"
	android:padding="16dp">
7	<ImageView
8	android:id="@+id/alatImage2"
9	android:layout_width="match_parent"
10	android:layout_height="200dp"
	android:scaleType="centerCrop"
	android:contentDescription="Gambar Alat" />
11	<TextView
	android:id="@+id/namaAlat"
	android:layout_width="match_parent"
	android:layout_height="wrap_content"
	android:text="Nama Alat"
	android:textSize="24sp"
	android:textStyle="bold"
	android:gravity="center"
	android:layout_marginTop="12dp" />
	<TextView
	android:id="@+id/deskripsiText"
	android:layout_width="match_parent"
	android:layout_height="wrap_content"
	android:text="Deskripsi alat gym"
	android:textSize="16sp"
	android:layout_marginTop="12dp" />
	<ImageView
	android:id="@+id/Gerakan"
	android:layout_width="match_parent"
	android:layout_height="200dp"
	android:layout_marginTop="16dp"
	android:scaleType="fitCenter"
	android:contentDescription="GIF Gerakan" />

```
</LinearLayout> </ScrollView>
```

Table 10. Source Code Jawaban Soal 1

7. AlatGymAdapter.kt

```

1 package com.example.zennfit import android.view.LayoutInflater
  import android.view.ViewGroup
2 import android.content.Intent
  import android.net.Uri
3 import androidx.recyclerview.widget.RecyclerView
  import com.example.zennfit.databinding.ListItemBinding
4
5 class AlatGymAdapter(
    private val data: List<MainActivity.AlatGym>,
6    private val onClick: (MainActivity.AlatGym) -> Unit
7 ) : RecyclerView.Adapter<AlatGymAdapter.ViewHolder>() {
8
9     class ViewHolder(val binding: ListItemBinding) :
  RecyclerView.ViewHolder(binding.root)
10
11     override fun onCreateViewHolder(parent: ViewGroup, viewType:
  Int): ViewHolder {
    val binding =
  ListItemBinding.inflate(LayoutInflater.from(parent.context), parent,
  false)
    return ViewHolder(binding)
  }
12
13     override fun onBindViewHolder(holder: ViewHolder, position: Int)
  {
    val alat = data[position]
    holder.binding.namaAlat.text = alat.nama
14
  holder.binding.alatImageList.setImageResource(alat.gambarResId)
    holder.binding.descSingkat.text = alat.descSingkat
    holder.binding.viewButton.setOnClickListener {
    onClick(alat)
  }
15
  holder.binding.browserButton.setOnClickListener {
    val context = holder.itemView.context
    val intent = Intent(
      Intent.ACTION_VIEW,
  Uri.parse("https://www.google.com/search?q=${Uri.encode(alat.nama)}")
    )
    context.startActivity(intent)
  }
  }
  }

```

	<pre> } override fun getItemCount(): Int = data.size } </pre>
--	---

Table 11. Source Code Jawaban Soal 11

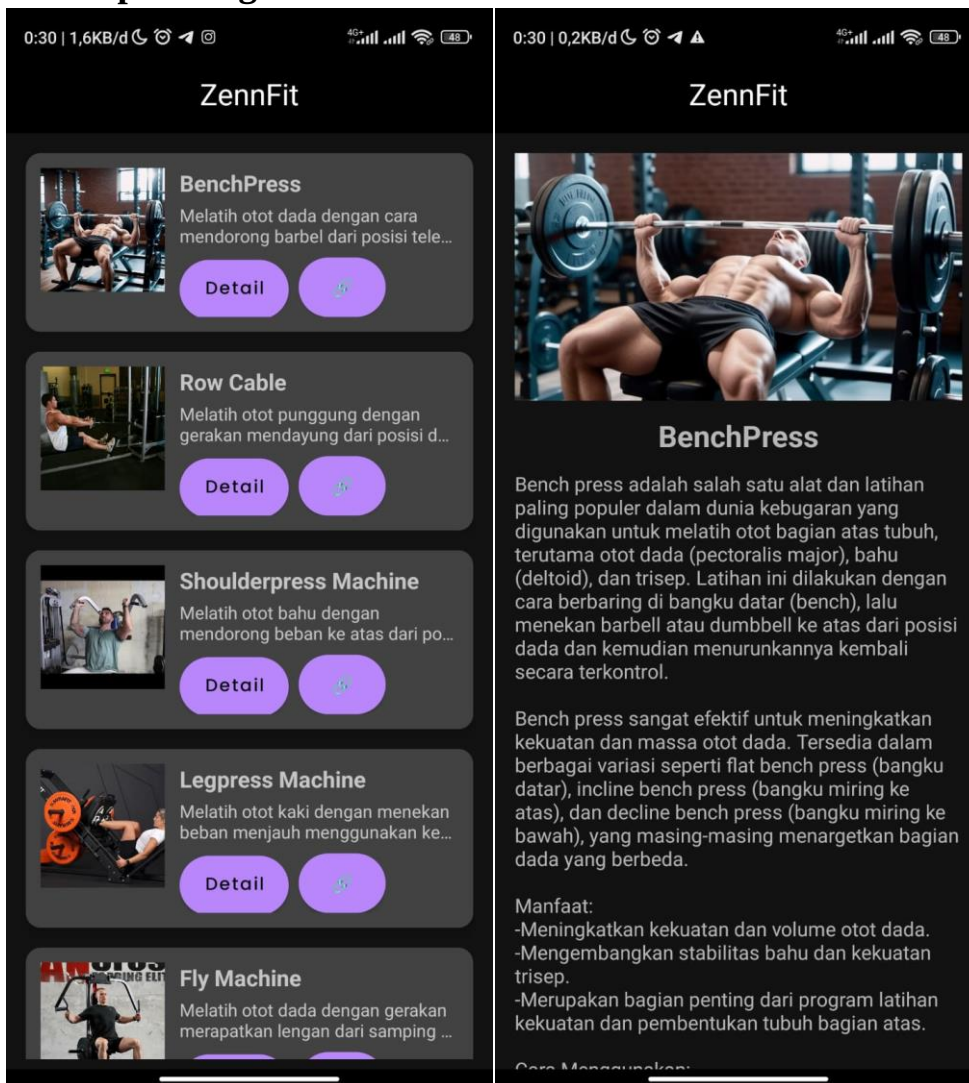
8. List_item.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	card_view:cardCornerRadius="12dp"
9	card_view:cardElevation="6dp">
10	<pre> <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="horizontal" android:padding="12dp"> </pre>
11	<pre> <ImageView android:id="@+id/alatImageList" android:layout_width="100dp" android:layout_height="100dp" android:scaleType="centerCrop" android:contentDescription="Gambar alat gymm" /> </pre>
	<pre> <LinearLayout android:layout_width="0dp" android:layout_weight="1" android:layout_height="wrap_content" android:orientation="vertical" android:layout_marginStart="12dp"> </pre>
	<pre> <TextView android:id="@+id/namaAlat" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="Nama Alat" android:textSize="18sp" android:textStyle="bold" /> </pre>
	<pre> <TextView android:id="@+id/descSingkat" android:text="Deskripsi singkat alat di sini." android:textSize="14sp" </pre>

	<pre> android:layout_width="wrap_content" android:layout_height="wrap_content" android:layout_marginTop="4dp" android:maxLines="2" android:ellipsize="end" /> <LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content" android:orientation="horizontal"> <Button android:id="@+id/viewButton" android:text="Detail" android:layout_width="wrap_content" android:layout_height="wrap_content" android:layout_marginTop="8dp" android:layout_marginEnd="8dp" android:fontFamily="@font/poppins_medium" android:textAllCaps="false" android:background="@drawable/rounded_button"/> <Button android:id="@+id/browserButton" android:text="🔍" android:layout_width="70dp" android:layout_height="wrap_content" android:layout_marginTop="8dp" android:background="@drawable/rounded_button"/> </LinearLayout> </LinearLayout> </LinearLayout> </androidx.cardview.widget.CardView> </pre>
--	--

Table 12. Source Code Jawaban Soal 1

B. Output Program



Gambar 3. Screenshot hasil soal 1

C. Pembahasan

3. MainActivity.kt:

package com.example.zennfit ini menyatakan bahwa file ini adalah bagian dari package com.example.zennfit. Selanjutnya ada import library, android.os.Bundle: Digunakan untuk menyimpan dan memulihkan state saat Activity dibuat. AppCompatActivity: Kelas dasar untuk Activity yang kompatibel dengan banyak versi Android. ActivityMainBinding: Kelas ViewBinding otomatis yang dihasilkan berdasarkan activity_main.xml. class MainActivity :

`AppCompatActivity()` { ini entry point dari aplikasi dan merupakan turunan dari `AppCompatActivity`.

Selanjutnya ada data class untuk merepresentasikan satu item alat gym. nama: Nama alat (misalnya "Bench Press"), descSingkat: Ringkasan deskripsi (misalnya "Alat melatih dada"), deskripsi: Penjelasan lengkap, gambarResId: ID resource gambar (misalnya `R.drawable.benchpress`), gifResId: ID resource untuk animasi atau gerakan alat (misalnya `R.drawable.benchpress_gif`). `private lateinit var binding: ActivityMainBinding` digunakan untuk mengakses komponen UI dalam `activity_main.xml` tanpa `findViewById`. `lateinit` ini artinya akan diinisialisasi nanti sebelum digunakan. pada kode `binding = ActivityMainBinding.inflate(layoutInflater)` `setContentView(binding.root)`, bagian ini `inflate(layoutInflater)` akan membuat objek binding dari layout `activity_main.xml`. `binding.root` adalah root view yang akan ditampilkan di layar. Selanjutnya menampilkan fragment default yang hanya dijalankan jika Activity baru pertama kali dibuat (bukan setelah rotasi layar). `supportFragmentManager` memulai `FragmentTransaction`: `replace(...)` akan mengganti isi `fragment_container` di `activity_main.xml` dengan `HomeFragment`. `commit()` menjalankan transaksinya.

4. activity_main.xml

`LinearLayout`: Mengatur elemen UI secara vertikal (dari atas ke bawah). `xmlns:android`: Deklarasi namespace untuk atribut Android. `layout_width="match_parent"` dan `layout_height="match_parent"`: Mengisi seluruh lebar dan tinggi layar. `orientation="vertical"`: Menyusun anak-anaknya secara vertikal.

Masuk keheader app, `layout_width="match_parent"`: Mengisi seluruh lebar layar. `layout_height="wrap_content"`: Tingginya disesuaikan dengan isi (teks). `gravity="center"`: Menempatkan teks di tengah horizontal. `padding="16dp"`: Memberi jarak di dalam komponen. `text="ZennFit"`: Menampilkan teks `ZennFit`. `background="@color/black"`: Latar belakang hitam (dari resource warna lokal). `textColor="@android:color/white"`: Teks berwarna putih. `textSize="24sp"`: Ukuran teks 24 scale-independent pixels.

Selanjutnya tempat untuk menampung fragment seperti `HomeFragment`, `DetailFragment`, dll. `id="@+id/fragment_container"`: Memberikan ID supaya dapat diakses dari kode Java/Kotlin. `layout_width` dan `layout_height` diatur ke `match_parent`, artinya mengisi sisa ruang layar setelah `TextView` di atas.

5. HomeFragment.kt

`class HomeFragment : Fragment()` turunan dari `Fragment`, yang artinya UI ini tidak berdiri sendiri, melainkan dimuat di dalam Activity (`MainActivity`). `Fragment` ini menampilkan list alat gym yang tersimpan dalam `RecyclerView`. `private var _binding: FragmentHomeBinding? = null` bagian `_binding` adalah variabel untuk mengakses komponen layout XML menggunakan `ViewBinding`. `FragmentHomeBinding` adalah binding otomatis yang dibuat dari `fragment_home.xml`. `private val binding get() = _binding!!` memastikan akses tidak null.

private lateinit var alatList: List<MainActivity.AlatGym> Menyimpan daftar data AlatGym yang akan ditampilkan. AlatGym adalah data class yang didefinisikan di MainActivity, merepresentasikan tiap alat gym (nama, deskripsi singkat, deskripsi panjang, gambar, dan gif). Di dalam onCreateView ada `_binding = FragmentHomeBinding.inflate(inflater, container, false)` return `binding.root` ini akan meng-inflate layout XML menjadi tampilan (View). Menggunakan ViewBinding untuk mengakses UI yang didefinisikan di `fragment_home.xml`.

Selanjutnya di dalam onCreateView disini membuat list berisi 5 alat gym. Setiap item berisi: nama: Nama alat, descSingkat: Deskripsi ringkas, deskripsi: Deskripsi panjang (diambil dari strings.xml), gambarResId: Gambar alat, gifResId: Gambar atau animasi tambahan. `binding.recyclerView.layoutManager = LinearLayoutManager(requireContext())` Mengatur RecyclerView supaya tampil secara vertikal (default) menggunakan LinearLayoutManager.

`binding.recyclerView.adapter = AlatGymAdapter(alatList) { alat -> ... }` Adapter dikustomisasi (AlatGymAdapter) untuk menampilkan data dan handle event klik pada tombol navigasi (ke detail). Callback `{ alat -> ... }` menangani klik navigasi ke detail. Lalu masuk navigasi ke DetailFragment saat diklik tombol navigasi maka data alat dikirim ke DetailFragment lewat Bundle menggunakan arguments. Selanjutnya Melakukan navigasi ke DetailFragment, mengganti fragment yang sedang tampil dan menambahkan ke backstack agar bisa kembali dengan tombol back. `onDestroyView()` ini mencegah memory leak dengan menghapus referensi ke binding saat fragment dihancurkan.

6. Fragment_Home.xml

`<LinearLayout ... >` ini adalah container yang menyusun elemen anaknya secara vertikal (atas ke bawah). `layout_width="match_parent"` dan `layout_height="match_parent"`: Layout ini akan mengisi seluruh layar pada lebar dan tinggi. `android:orientation="vertical"`: Elemen-elemen di dalam LinearLayout akan disusun secara vertikal.

`<androidx.recyclerview.widget.RecyclerView ... />` ini adalah komponen RecyclerView, digunakan untuk menampilkan daftar data yang dapat di-scroll secara efisien. `android:id="@+id/recyclerView"`: ID yang digunakan untuk mengakses RecyclerView dari kode Kotlin di HomeFragment. `layout_width="match_parent"` dan `layout_height="match_parent"`:. RecyclerView akan mengisi seluruh ruang dari LinearLayout, baik lebar maupun tinggi. `android:padding="8dp"`: Memberi jarak 8dp di sekeliling RecyclerView, agar isi tidak mepet ke pinggir layar. `android:contentDescription="Daftar alat gym"`: Digunakan untuk aksesibilitas, agar pembaca layar bisa mengenali bahwa ini adalah daftar alat gym.

7. DetailFragment.kt

Didalam kode ini memakai `private var _binding: FragmentDetailBinding? = null`, menggunakan ViewBinding agar bisa mengakses elemen UI dari XML tanpa `findViewById`. `private val binding get() = _binding!!` _binding ini dibersihkan di `onDestroyView()` untuk

mencegah memory leak. Selanjutnya ada companion object yaitu membuat method newInstance() untuk mengemas data ke dalam Bundle dan mengatur sebagai arguments pada fragment. Memudahkan pemanggilan fragment ini dari luar (misalnya dari adapter atau fragment lain).

Menampilkan data alatnya, data diambil dari arguments (Bundle) yang dikirim sebelumnya saat berpindah dari list. Lalu ada set data ke UI, binding.namaAlat.text = nama, binding.deskripsiText.text = deskripsi, binding.alatImage2.setImageResource(gambar), Glide.with(this).asGif().load(gif).into(binding.Gerakan). nama dan deskripsi ditampilkan ke TextView. gambar ditampilkan ke ImageView. gif ditampilkan menggunakan Glide (asGif()), karena Android ImageView biasa tidak mendukung GIF secara default. Terakhir ada Membersihkan binding ini hanya praktik standar agar binding tidak menampung referensi view yang sudah hancur (mencegah memory leak).

8. Fragment_detail.xml

ScrollView: Agar tampilan bisa di-scroll, terutama jika konten lebih tinggi dari layar (penting saat orientasi berubah). LinearLayout: Digunakan untuk menyusun item secara vertikal (orientation="vertical"). ImageView gambar alatnya menampilkan gambar statis alat gym (misal: foto alat). scaleType="centerCrop": Memotong gambar agar memenuhi seluruh lebar, tetap menjaga rasio aspek. contentDescription: : Penting untuk aksesibilitas (dibaca oleh screen reader). TextView nama alatnya menampilkan nama alat gym (judul). Font besar (24sp), tebal, dan tengah (gravity="center") agar terlihat seperti header.

Selanjutnya ke TextView Deskripsi alatnya menampilkan deskripsi panjang alat gym. Lebih kecil dari judul (16sp) dan diletakkan setelah nama alat. ImageView untuk gif gambar menampilkan gerakan alat dalam bentuk GIF. scaleType="fitCenter" agar GIF ditampilkan penuh namun tetap proporsional. Dimuat menggunakan Glide di DetailFragment.kt.

9. AlatGymAdapter.kt

class AlatGymAdapter(private val data: List<MainActivity.AlatGym>, data: daftar item alat gym yang akan ditampilkan. private val onClick: (MainActivity.AlatGym) -> Unit) : RecyclerView.Adapter<AlatGymAdapter.ViewHolder>() {
onClick: lambda function (callback) untuk meng-handle klik tombol detail.
class ViewHolder(val binding: ListItemBinding) : RecyclerView.ViewHolder(binding.root) menyimpan binding ke tampilan item list (list_item.xml). Menggunakan ViewBinding agar lebih aman dan efisien saat mengakses komponen layout.
val alat = data[position] mengambil item berdasarkan posisi dari List<AlatGym>. Lalu Menampilkan nama alat, gambar, dan deskripsi singkat di layout item list dengan binding.

holder.binding.viewButton.setOnClickListener { onClick(alat)} Saat tombol "Lihat" ditekan, memanggil fungsi onClick() dari constructor adapter. Diimplementasikan di HomeFragment untuk navigasi ke DetailFragment. Lalu ke button browser akan membuka

browser dan melakukan pencarian Google berdasarkan nama alat. override fun getItemCount(): Int = data.size akan mengembalikan jumlah item pada list.

10. List_item.xml

```
<androidx.cardview.widget.CardView,  
android:layout_margin="8dp" card_view:cardCornerRadius="12dp"  
card_view:cardElevation="6dp">
```

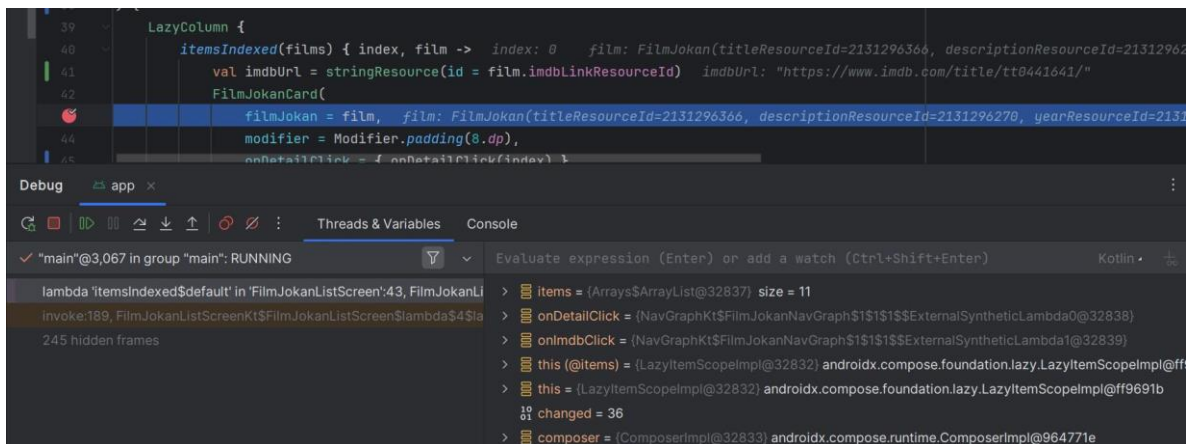
membuat efek kartu mengambang (elevasi) dengan sudut bulat. Memberi kesan UI modern dan rapi. pada linera layout, android:orientation="horizontal" membuat isi berjejer ke kanan: Gambar di kiri, teks dan tombol di kanan. ImageView alatImageView Menampilkan gambar alat gym. centerCrop: gambar dipotong agar pas dan tidak melebar/mengecil secara tidak proporsional. TextView namaAlat menampilkan nama alat gym. Teks tebal dan besar agar terlihat seperti judul. TextView DescSingkat menampilkan deskripsi singkat alat gym. Jika terlalu panjang, hanya ditampilkan 2 baris dan dipotong dengan Tombol detail saat ditekan, akan membuka DetailFragment. Memakai background kustom rounded_button agar tampil modern (biasanya berbentuk bulat dengan warna khusus). Tombol browser saat ditekan, akan membuka Google search dengan nama alat gym. Tombol lebih pendek (70dp) dan hanya berisi emoji/link icon. saya juga nambahkan styling di @font/poppins_medium: font khusus agar teks terlihat lebih profesional. @drawable/rounded_button: file drawable kustom yang biasanya digunakan untuk membuat tombol dengan sudut bulat dan warna tertentu.

MODUL 4 : ViewModel & Debugging

SOAL 1

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data
 - b. tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - c. Gunakan ViewModelFactory dalam pembuatan ViewModel
 - d. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out
2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



A. Source Code

1. Alat Gym.kt

1	<code>package com.example.zennfit</code>
2	<code>data class AlatGym(</code>
3	<code> val id: Int,</code>
4	<code> val nama: String,</code>
5	<code> val deskripsiResId: Int,</code>
	<code> val imageResId: Int,</code>
	<code> val gifResId: Int,</code>
	<code>)</code>

Table 13. Source Code Jawaban Soal 1

2. AlatGymViewModel

1	<code>package com.example.zennfit</code>
2	<code>import androidx.lifecycle.ViewModel</code>
	<code>import kotlinx.coroutines.flow.MutableStateFlow</code>
3	<code>import kotlinx.coroutines.flow.StateFlow</code>
4	<code>import kotlinx.coroutines.flow.asStateFlow</code>
5	<code>class AlatGymViewModel : ViewModel() {</code>
	<code> private val _alatList =</code>
6	<code>MutableStateFlow<List<AlatGym>>(emptyList())</code>
	<code> val alatList: StateFlow<List<AlatGym>> = _alatList.asStateFlow()</code>
7	<code> init {</code>
8	<code> loadAlatGym()</code>
9	<code> }</code>
	<code> private fun loadAlatGym() {</code>
10	<code> val data = listOf(</code>
	<code> AlatGym(1, "BenchPress", R.string.desc_benchpress,</code>
11	<code>R.drawable.benchpress, R.drawable.incline_benchpress),</code>
	<code> AlatGym(2, "Row Cable", R.string.desc_row,</code>
	<code>R.drawable.cablerow, R.drawable.rowbro),</code>
	<code> AlatGym(3, "Shoulderpress Machine",</code>
	<code>R.string.desc_shoulder, R.drawable.shoulderpress,</code>
	<code>R.drawable.shoulderpress_machine),</code>
	<code> AlatGym(4, "Legpress Machine", R.string.desc_legpress,</code>
	<code>R.drawable.legpress, R.drawable.legpressmachine),</code>
	<code> AlatGym(5, "Fly Machine", R.string.desc_fly,</code>
	<code>R.drawable.fly, R.drawable.fly_machine)</code>
	<code>)</code>
	<code> _alatList.value = data</code>
	<code> }</code>
	<code>}</code>

Table 14. Source Code Jawaban Soal 1

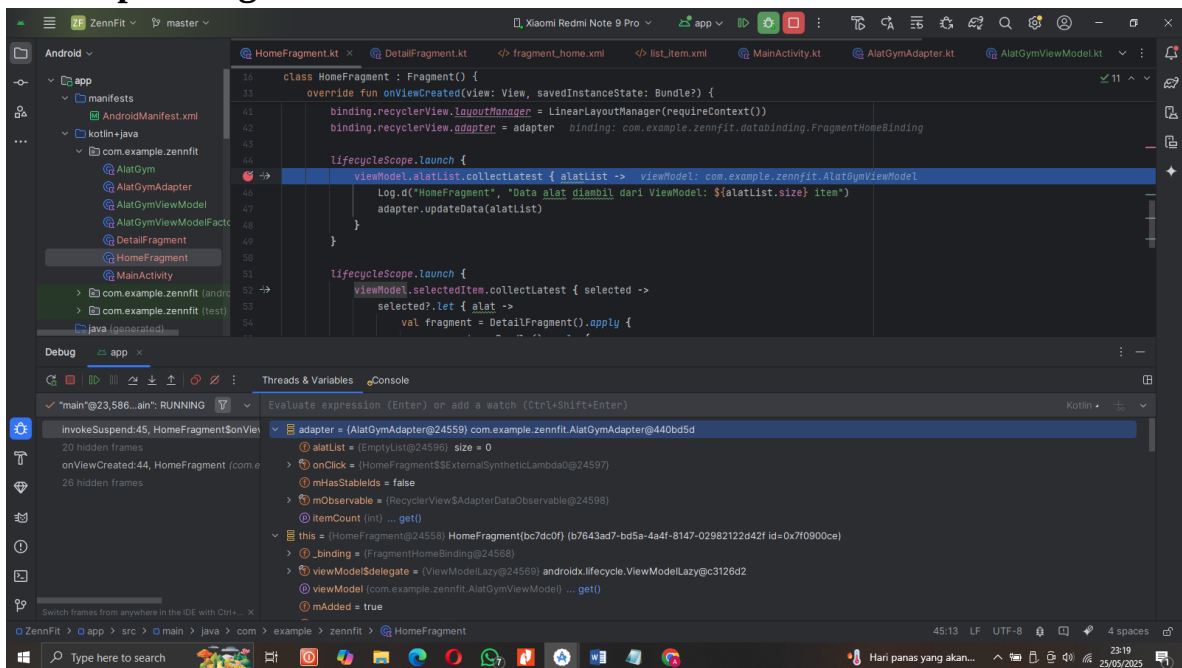
3. AlatGymViewModel

1	<code>package com.example.zennfit</code>
---	--

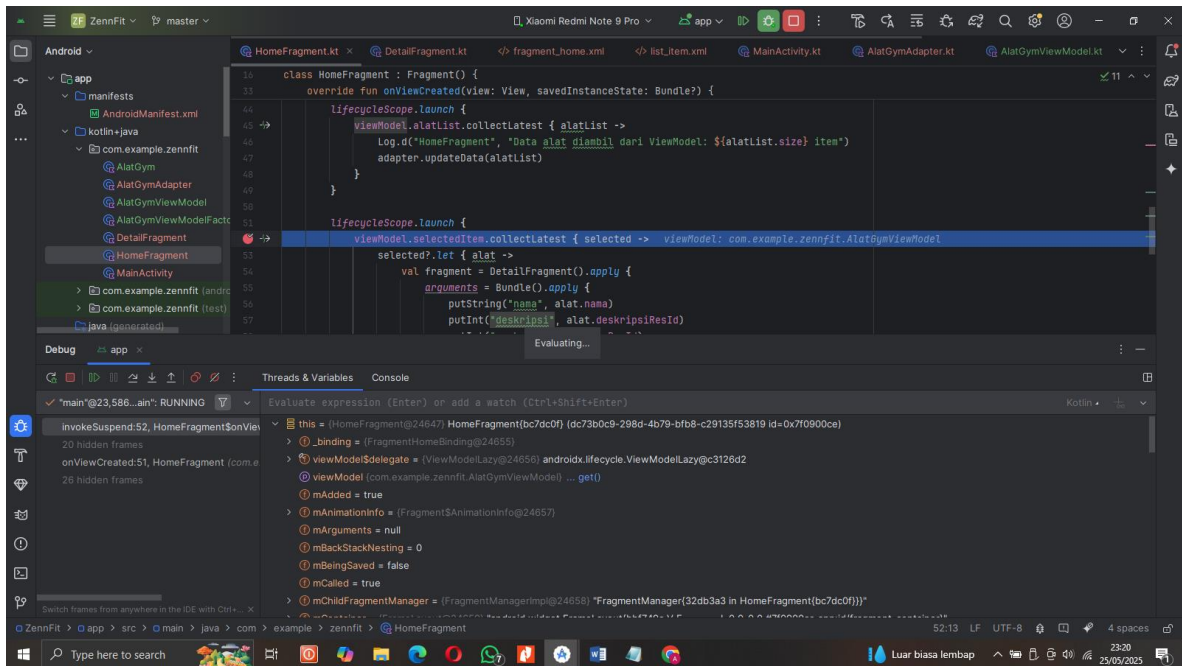
2	<code>import androidx.lifecycle.ViewModel</code>
3	<code>import androidx.lifecycle.ViewModelProvider</code>
4	<code>class AlatGymViewModelFactory : ViewModelProvider.Factory {</code>
5	<code> override fun <T : ViewModel> create(modelClass: Class<T>): T {</code>
6	<code> if</code>
7	<code> (modelClass.isAssignableFrom(AlatGymViewModel::class.java)) {</code>
8	<code> return AlatGymViewModel() as T</code>
	<code> } else {</code>
	<code> throw IllegalArgumentException("Unknown ViewModel class")</code>
	<code> }</code>
	<code>}</code>

Table 15. Source Code Jawaban Soal 1

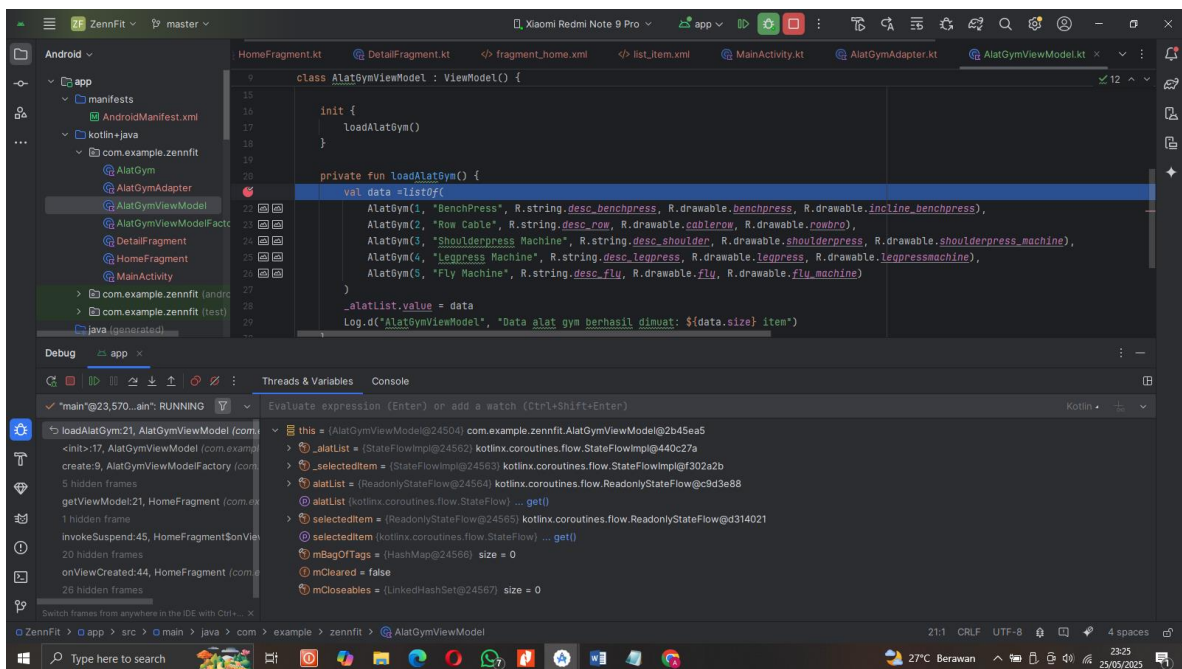
B. Output Program



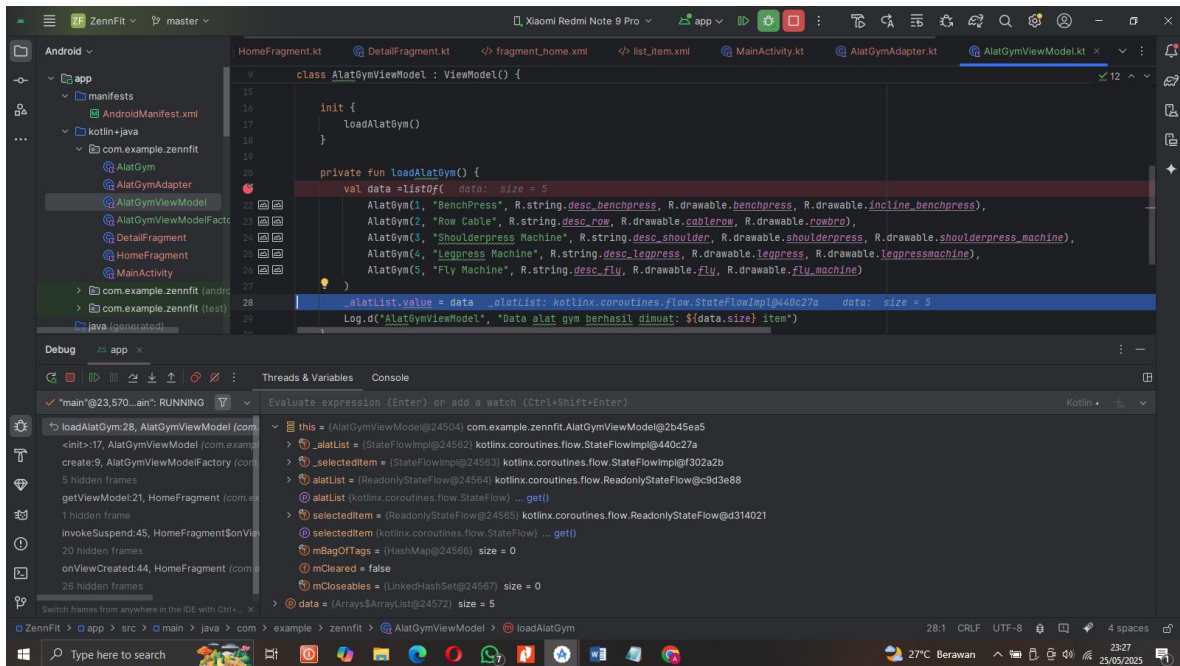
Gambar 4 Screenshot hasil soal 1



Gambar 5 Screenshot hasil soal 1



Gambar 6 Screenshot hasil soal 1



Gambar 7 Screenshot hasil soal 1

C. Pembahasan

Debugger adalah alat di Android Studio yang membantu developer melihat jalannya aplikasi secara rinci saat runtime, menghentikan eksekusi di titik tertentu (breakpoint), dan memeriksa nilai variabel serta alur program.

Cara menggunakan Debugger:

1. Letakkan breakpoint dengan klik kiri di margin kiri editor.
2. Jalankan aplikasi dalam mode Debug (klik ikon bug atau Shift+F9).
3. Ketika eksekusi sampai breakpoint, aplikasi berhenti dan developer dapat memeriksa variabel, thread, dan call stack.
4. Gunakan fitur stepping untuk menelusuri kode.

Fitur stepping:

1. **Step Into:** Masuk ke dalam fungsi/method yang dipanggil, melihat implementasinya baris demi baris.
2. **Step Over:** Lewati fungsi/method yang dipanggil, langsung lanjut ke baris berikutnya di fungsi saat ini.
3. **Step Out:** Keluar dari fungsi/method saat ini dan kembali ke fungsi pemanggil.

Application Class

Application class adalah kelas dasar yang mewakili keseluruhan aplikasi Android. Kelas ini dijalankan pertama kali sebelum komponen lain (Activity, Service, dll). Fungsinya untuk menginisialisasi resource atau konfigurasi yang bersifat global, misalnya:

- Inisialisasi library seperti database, logging, dependency injection (Dagger/Hilt).
- Menyimpan state atau data yang ingin diakses secara global oleh seluruh komponen aplikasi.
- Mengelola lifecycle aplikasi secara keseluruhan.

Biasanya developer membuat subclass Application untuk melakukan setup awal dan mendaftarkannya di AndroidManifest.xml. Dengan Application class, kode konfigurasi tidak perlu diulang-ulang di setiap Activity atau Fragment.

11. AlatGym.kt

Isi dalamnya adalah deklarasi dari sebuah data bernama AlatGym, bisa dilihat ada val id: Int, val nama: String, val deskripsiResId: Int, val imageResId: Int, val gifResId: Int,

12. AlatGymViewModel

ViewModel ini tempat menyimpan data list alat gym yang sudah dimodifikasi yang mana sebelumnya disimpan di dalam Home fragment. Jadi deklarasi class AlatGymViewModel yang mana ini adalah subclass dari ViewModel. Lalu ada property privat _alatlist dan property public alatList yang hanya bisa dibaca dari luar. Lalu menaggil blok LoadAlatGym untuk Mengisi data alat gymnya . Fungsi ini membuat listalat gym secara manual lalu menyimpannya kedalam _alatList.

13. AlatGymViewModelFactory

Kode ini masih default karena tidak ada parameter pada viewModel.

MODUL 5 : Connect to the Internet

SOAL 1

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
 - b. Gunakan KotlinX Serialization sebagai library JSON.
 - c. Gunakan library seperti Coil atau Glide untuk image loading.
 - d. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
 - e. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
 - f. Gunakan caching strategy pada Room..
 - g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. HomeFragment.kt

```
1 package com.example.zennfit
2
3 import android.os.Bundle
4 import android.util.Log
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import androidx.fragment.app.Fragment
9 import androidx.fragment.app.viewModels
10 import androidx.lifecycle.LifecycleScope
11 import androidx.recyclerview.widget.LinearLayoutManager
12 import com.example.zennfit.databinding.FragmentHomeBinding
13 import kotlinx.coroutines.flow.collectLatest
14 import kotlinx.coroutines.launch
15
16 import com.example.zennfit.network.ApiClient
17 import com.example.zennfit.data.AppDatabase
18 import com.example.zennfit.data.AlatGymRepository
19 import com.example.zennfit.model.getLocalizedDescription
20 import com.example.zennfit.model.getLocalizedName
21 import com.example.zennfit.model.getMainImageUrl
```

```

import com.example.zennfit.model.getMainVideoUrl

class HomeFragment : Fragment() {

    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    private val viewModel: AlatGymViewModel by viewModels {
        Log.d("HomeFragmentInit", "Starting ViewModel
initialization.")
        try {
            val exerciseInfoDao =
AppDatabase.getDatabase(requireContext()).exerciseInfoDao()
            Log.d("HomeFragmentInit", "ExerciseInfoDao initialized.")
            val wgerApiService = ApiClient.wgerApiService
            Log.d("HomeFragmentInit", "WgerApiService initialized.")
            val repository = AlatGymRepository(wgerApiService,
exerciseInfoDao)
            Log.d("HomeFragmentInit", "AlatGymRepository
initialized.")
            AlatGymViewModelFactory(repository)
        } catch (e: Exception) {
            Log.e("HomeFragmentInit", "Error during ViewModel
initialization: ${e.message}", e)
            throw e
        }
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container,
false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d("HomeFragment", "onViewCreated called.")

        val adapter = AlatGymAdapter(emptyList()) { exerciseInfo ->
            viewModel.onItemClicked(exerciseInfo)
            Log.d("HomeFragment", "Tombol item diklik:
${exerciseInfo.getLocalizedName()}")
        }

        binding.recyclerView.layoutManager =
LinearLayoutManager(requireContext())
        binding.recyclerView.adapter = adapter
    }
}

```

```

        lifecycleScope.launch {
            viewModel.alatList.collectLatest { exerciseInfoList ->
                Log.d("HomeFragment", "Data alat diambil dari
ViewModel: ${exerciseInfoList.size} item")
                Log.d("HomeFragment", "Calling adapter.updateData()
with ${exerciseInfoList.size} items.")
                adapter.updateData(exerciseInfoList)
            }
        }

        lifecycleScope.launch {
            viewModel.selectedItem.collectLatest { selected ->
                selected?.let { exerciseInfo ->
                    val fragment = DetailFragment().apply {
                        arguments = Bundle().apply {
                            putString("nama",
exerciseInfo.getLocalizedName())
                            putString("deskripsi",
exerciseInfo.getLocalizedDescription())
                            putString("imageUrl",
exerciseInfo.getMainImageUrl())
                            putString("videoUrl",
exerciseInfo.getMainVideoUrl())
                        }
                    }

                    parentFragmentManager.beginTransaction()
                        .replace(R.id.fragment_container, fragment)
                        .addToBackStack(null)
                        .commit()
                    viewModel.clearSelectedItem()
                    Log.d("HomeFragment", "Berpindah ke
DetailFragment: ${exerciseInfo.getLocalizedName()}")
                }
            }
        }

        override fun onDestroyView() {
            super.onDestroyView()
            _binding = null
        }
    }
}

```

Table 16. Source Code Jawaban Soal 1

2. DetailFragment.kt

```

1 package com.example.zennfit
2 import android.os.Bundle

```

```

3 import android.view.LayoutInflater
import android.view.View
4 import android.view.ViewGroup
import androidx.fragment.app.Fragment
5 import com.bumptech.glide.Glide
import com.example.zennfit.databinding.FragmentDetailBinding
6 import android.util.Log
7
import com.google.android.exoplayer2.ExoPlayer
8 import com.google.android.exoplayer2.MediaItem
9
10 class DetailFragment : Fragment() {
11     private var _binding: FragmentDetailBinding? = null
        private val binding get() = _binding!!

        private var player: ExoPlayer? = null
        private var playWhenReady = true
        private var playbackPosition = 0L

        companion object {
            fun newInstance(nama: String, deskripsi: String, imageUrl:
String?, videoUrl: String?): DetailFragment {
                val fragment = DetailFragment()
                val bundle = Bundle().apply {
                    putString("nama", nama)
                    putString("deskripsi", deskripsi)
                    putString("imageUrl", imageUrl)
                    putString("videoUrl", videoUrl)
                }
                fragment.arguments = bundle
                return fragment
            }
        }

        override fun onCreateView(
            inflater: LayoutInflater, container: ViewGroup?,
            savedInstanceState: Bundle?
        ): View {
            _binding = FragmentDetailBinding.inflate(inflater, container,
false)
            return binding.root
        }

        override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
            super.onViewCreated(view, savedInstanceState)

            val nama = arguments?.getString("nama")
            val deskripsi = arguments?.getString("deskripsi") ?:
"Deskripsi tidak tersedia."
            val imageUrl = arguments?.getString("imageUrl")

```

```

        val videoUrl = arguments?.getString("videoUrl")

        Log.d("DetailFragment", "Menampilkan detail untuk: $nama")
        Log.d("DetailFragment", "Deskripsi: $deskripsi")
        Log.d("DetailFragment", "Image URL: $imageUrl")
        Log.d("DetailFragment", "Video URL: $videoUrl")

        activity?.title = nama
        binding.namaAlat.text = nama
        binding.deskripsiText.text = deskripsi

        if (!imageUrl.isNullOrEmpty()) {
            binding.alatImage2.visibility = View.VISIBLE
            Glide.with(this)
                .load(imageUrl)
                .placeholder(R.drawable.placeholder_image)
                .error(R.drawable.placeholder_image)
                .into(binding.alatImage2)
        } else {
            binding.alatImage2.visibility = View.GONE
        }

        if (!videoUrl.isNullOrEmpty()) {
            binding.videoPlayerView.visibility = View.VISIBLE
            initializePlayer(videoUrl)
        } else {
            binding.videoPlayerView.visibility = View.GONE
            Log.d("DetailFragment", "No video URL provided,
PlayerView hidden.")
        }
    }

    private fun initializePlayer(videoUrl: String) {
        if (player == null) {
            player = ExoPlayer.Builder(requireContext())
                .build()
            binding.videoPlayerView.player = player
        }

        val mediaItem = MediaItem.fromUri(videoUrl)
        player?.setMediaItem(mediaItem)
        player?.seekTo(playbackPosition)
        player?.playWhenReady = playWhenReady
        player?.prepare()
        player?.repeatMode = ExoPlayer.REPEAT_MODE_ALL
        Log.d("DetailFragment", "ExoPlayer initialized and trying to
play: $videoUrl")
    }

    override fun onStart() {
        super.onStart()
        val videoUrl = arguments?.getString("videoUrl")
        if (!videoUrl.isNullOrEmpty() && player == null) {

```

	<pre> initializePlayer(videoUrl) } } override fun onResume() { super.onResume() player?.playWhenReady = true Log.d("DetailFragment", "ExoPlayer onResume: playWhenReady = true") } override fun onPause() { super.onPause() player?.playWhenReady = false playbackPosition = player?.currentPosition ?: 0L playWhenReady = player?.playWhenReady ?: true Log.d("DetailFragment", "ExoPlayer onPause: playWhenReady = \$playWhenReady, position = \$playbackPosition") } override fun onStop() { super.onStop() releasePlayer() Log.d("DetailFragment", "ExoPlayer onStop: Player released.") } private fun releasePlayer() { if (player != null) { player?.release() player = null Log.d("DetailFragment", "ExoPlayer resources released.") } } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	--

Table 17. Source Code Jawaban Soal 1

3. AlatGymViewModel

1	<code>package com.example.zennfit</code>
2	<code>import androidx.lifecycle.ViewModel</code>
3	<code>import androidx.lifecycle.viewModelScope</code>
	<code>import com.example.zennfit.data.AlatGymRepository</code>


```

4 import com.example.zennfit.model.ExerciseInfo
import com.example.zennfit.model.getLocalizedName
5 import com.example.zennfit.model.getLocalizedName
import com.example.zennfit.model.getLocalizedName
6 import com.example.zennfit.model.getLocalizedName
import com.example.zennfit.model.getLocalizedName
7
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
9 import kotlinx.coroutines.launch
import android.util.Log
10
11 class AlatGymViewModel(
    private val repository: AlatGymRepository
) : ViewModel() {

    private val _alatList =
MutableStateFlow<List<ExerciseInfo>>(emptyList())
    val alatList: StateFlow<List<ExerciseInfo>> =
    _alatList.asStateFlow()

    private val _selectedItem = MutableStateFlow<ExerciseInfo?>(null)
    val selectedItem: StateFlow<ExerciseInfo?> =
    _selectedItem.asStateFlow()

    private val curatedExerciseIds = listOf(
        73, // Bench Press
        272, // Hammercurls
        465,
        567,
        194,
        475
    )

    init {
        Log.d("AlatGymViewModel", "ViewModel init block executed.")
        loadCuratedExercises()
    }

    private fun loadCuratedExercises() {
        Log.d("AlatGymViewModel", "loadCuratedExercises() called to
fetch specific IDs.")
        viewModelScope.launch {
            val loadedExercises = mutableListOf<ExerciseInfo>()
            for (id in curatedExerciseIds) {
                Log.d("AlatGymViewModel", "Attempting to fetch
exercise with ID: $id")
                val exercise = repository.getExerciseInfoById(id)
                if (exercise != null) {
                    val hasImage =
!exercise.getMainImageUrl().isNullOrEmpty()
                    val hasDescription =
!(exercise.getLocalizedName() == "Deskripsi tidak tersedia."

```

```

|| exercise.getLocalizedDescription().isBlank())
        val hasVideo =
!exercise.getMainVideoUrl().isNullOrEmpty()

        if (hasImage && hasDescription) {
            loadedExercises.add(exercise)
            Log.d("AlatGymViewModel", "Successfully added
complete exercise ID $id: ${exercise.getLocalizedName()} | HasImg:
$hasImage | HasDesc: $hasDescription | HasVid: $hasVideo")
        } else {
            Log.w("AlatGymViewModel", "Exercise ID $id
(${exercise.getLocalizedName()}) is not 'complete' enough (missing
img/desc). Not adding to list. HasImg: $hasImage | HasDesc:
$hasDescription | HasVid: $hasVideo")
        }

    } else {
        Log.w("AlatGymViewModel", "Failed to fetch
exercise with ID: $id. Not adding to list.")
    }
}
_alatList.value = loadedExercises
Log.d("AlatGymViewModel", "Finished loading curated
exercises. Displaying ${loadedExercises.size} items.")
}

fun onItemClick(item: ExerciseInfo) {
    _selectedItem.value = item
    Log.d("AlatGymViewModel", "Item diklik:
${item.getLocalizedName()}")
}

fun clearSelectedItem() {
    _selectedItem.value = null
    Log.d("AlatGymViewModel", "Selected item cleared.")
}

fun refreshData() {
    Log.d("AlatGymViewModel", "refreshData() called. Reloading
curated exercises.")
    loadCuratedExercises()
}
}

```

Table 18. Source Code Jawaban Soal 1

4. AlatGymViewModelFactory

```

1 package com.example.zennfit

import androidx.lifecycle.ViewModel

```

2	<code>import androidx.lifecycle.ViewModelProvider</code>
3	<code>import com.example.zennfit.data.AlatGymRepository</code>
4	<code>class AlatGymViewModelFactory(</code>
5	<code> private val repository: AlatGymRepository</code>
6	<code>) : ViewModelProvider.Factory {</code>
7	<code> override fun <T : ViewModel> create(modelClass: Class<T>): T {</code>
8	<code> if</code>
9	<code>(modelClass.isAssignableFrom(AlatGymViewModel::class.java)) {</code>
10	<code> @Suppress("UNCHECKED_CAST")</code>
11	<code> return AlatGymViewModel(repository) as T</code>
	<code> }</code>
	<code> throw IllegalArgumentException("Unknown ViewModel class")</code>
	<code> }</code>
	<code>}</code>

Table 19. Source Code Jawaban Soal 1

5. WgerApiService

1	<code>package com.example.zennfit.network</code>
2	<code>import com.example.zennfit.model.WgerExerciseInfoResponse</code>
3	<code>import com.example.zennfit.model.ExerciseInfo</code>
4	<code>import retrofit2.Response</code>
5	<code>import retrofit2.http.GET</code>
6	<code>import retrofit2.http.Path</code>
7	<code>import retrofit2.http.Query</code>
8	<code>interface WgerApiService {</code>
9	<code> @GET("api/v2/exerciseinfo/")</code>
10	<code> suspend fun getExerciseInfo(</code>
11	<code> @Query("search") query: String? = null</code>
	<code>) : Response<WgerExerciseInfoResponse></code>
	<code></code>
	<code> @GET("api/v2/exerciseinfo/{id}/")</code>
	<code> suspend fun getExerciseInfoById(</code>
	<code> @Path("id") id: Int</code>
	<code>) : Response<ExerciseInfo></code>
	<code>}</code>

Table 20. Source Code Jawaban Soal 1

6. ApiClient

1	<code>package com.example.zennfit.network</code>
2	<code>import okhttp3.OkHttpClient</code>
3	<code>import okhttp3.logging.HttpLoggingInterceptor</code>
4	<code>import retrofit2.Retrofit</code>

4	<code>import retrofit2.converter.gson.GsonConverterFactory</code>
5	<code>import java.util.concurrent.TimeUnit</code>
6	<code>object ApiClient {</code>
7	<code> private const val BASE_URL = "https://wger.de/"</code>
8	<code> private val loggingInterceptor = HttpLoggingInterceptor().apply {</code>
9	<code> level = HttpLoggingInterceptor.Level.BODY</code>
10	<code> } private val okHttpClient = OkHttpClient.Builder()</code>
11	<code> .addInterceptor(loggingInterceptor) .connectTimeout(30, TimeUnit.SECONDS) .readTimeout(30, TimeUnit.SECONDS) .writeTimeout(30, TimeUnit.SECONDS) .build() private val retrofit: Retrofit by lazy { Retrofit.Builder() .baseUrl(BASE_URL) .client(okHttpClient) .addConverterFactory(GsonConverterFactory.create()) .build() } val wgerApiService: WgerApiService by lazy { retrofit.create(WgerApiService::class.java) } }</code>

Table 21. Source Code Jawaban Soal 1

7. WgerEquipmentInfoResponse

1	<code>package com.example.zennfit.model</code>
2	<code>import com.google.gson.annotations.SerializedName</code>
3	<code>data class WgerExerciseInfoResponse(</code>
4	<code> @SerializedName("count") val count: Int,</code>
5	<code> @SerializedName("next") val next: String?,</code>
6	<code> @SerializedName("previous") val previous: String?,</code>
7	<code> @SerializedName("results") val results: List<ExerciseInfo> //</code>
8	<code>Ganti List<Equipment> ke List<ExerciseInfo></code>
9	<code>)</code>

Table 22. Source Code Jawaban Soal 1

8. ExerciseInfo

1	<code>package com.example.zennfit.model</code>
2	<code>import com.google.gson.annotations.SerializedName</code>
3	<code>import android.text.Html</code>

```

3 import android.os.Build
4 data class ExerciseInfo(
5     @SerializedName("id") val id: Int,
6     @SerializedName("uuid") val uuid: String,
7     @SerializedName("created") val created: String?,
8     @SerializedName("last_update") val lastUpdate: String?,
9     @SerializedName("last_update_global") val lastUpdateGlobal:
10 String?,
11     @SerializedName("category") val category: Category?,
12     @SerializedName("muscles") val muscles: List<Muscle>?,
13     @SerializedName("muscles_secondary") val musclesSecondary:
14 List<Muscle>?,
15     @SerializedName("equipment") val equipment:
16 List<EquipmentDetail>?,
17     @SerializedName("license") val license: License?,
18     @SerializedName("license_author") val licenseAuthor: String?,
19     @SerializedName("images") val images: List<ExerciseImage>?,
20     @SerializedName("translations") val translations:
21 List<ExerciseTranslation>?,
22     @SerializedName("variations") val variations: Any?,
23     @SerializedName("videos") val videos: List<ExerciseVideo>?,
24     @SerializedName("author_history") val authorHistory:
25 List<String>?,
26     @SerializedName("total_authors_history") val totalAuthorsHistory:
27 List<String>?
28 )
29 data class Category(
30     @SerializedName("id") val id: Int,
31     @SerializedName("name") val name: String
32 )
33 data class Muscle(
34     @SerializedName("id") val id: Int,
35     @SerializedName("name") val name: String,
36     @SerializedName("name_en") val nameEn: String?,
37     @SerializedName("is_front") val isFront: Boolean?,
38     @SerializedName("image_url_main") val imageUrlMain: String?,
39     @SerializedName("image_url_secondary") val imageUrlSecondary:
40 String?
41 )
42 data class EquipmentDetail(
43     @SerializedName("id") val id: Int,
44     @SerializedName("name") val name: String
45 )
46 data class License(
47     @SerializedName("id") val id: Int,
48     @SerializedName("full_name") val fullName: String,
49     @SerializedName("short_name") val shortName: String,
50     @SerializedName("url") val url: String
51 )
52 data class ExerciseImage(
53     @SerializedName("id") val id: Int,

```

```

        @SerializedName("exercise_base") val exerciseBaseId: Int?,
        @SerializedName("image") val imageUrl: String,
        @SerializedName("is_main") val isMain: Boolean,
        @SerializedName("uuid") val uuid: String
    )

    data class ExerciseVideo(
        @SerializedName("id") val id: Int,
        @SerializedName("exercise_base") val exerciseBaseId: Int?,
        @SerializedName("video") val videoUrl: String,
        @SerializedName("is_main") val isMain: Boolean,
        @SerializedName("uuid") val uuid: String
    )

    data class ExerciseTranslation(
        @SerializedName("id") val id: Int,
        @SerializedName("uuid") val uuid: String,
        @SerializedName("name") val name: String,
        @SerializedName("exercise") val exerciseId: Int?,
        @SerializedName("description") val description: String,
        @SerializedName("created") val created: String?,
        @SerializedName("language") val languageId: Int,
        @SerializedName("aliases") val aliases: List<Any>?,
        @SerializedName("notes") val notes: List<Any>?,
        @SerializedName("license") val license: Int?,
        @SerializedName("license_title") val licenseTitle: String?,
        @SerializedName("license_object_url") val licenseObjectUrl:
String?,
        @SerializedName("license_author") val licenseAuthor: String?,
        @SerializedName("license_author_url") val licenseAuthorUrl:
String?,
        @SerializedName("license_derivative_source_url") val
licenseDerivativeSourceUrl: String?,
        @SerializedName("author_history") val authorHistory:
List<String>?
    )

    fun ExerciseInfo.getLocalizedName(languageId: Int = 2): String {
        val localizedTranslation = translations?.firstOrNull {
            it.languageId == languageId }

        return localizedTranslation?.name ?:
translations?.firstOrNull()?.name ?: "Nama tidak tersedia"
    }

    fun ExerciseInfo.getLocalizedDescription(languageId: Int = 2): String
    {
        val localizedTranslation = translations?.firstOrNull {
            it.languageId == languageId }

        val htmlDescription = localizedTranslation?.description
            ?: translations?.firstOrNull()?.description

        return if (!htmlDescription.isNullOrEmpty()) {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
                Html.fromHtml(htmlDescription,

```

	<pre> Html.FROM_HTML_MODE_COMPACT).toString() } else { @Suppress("DEPRECATION") Html.fromHtml(htmlDescription).toString() } } else { "Deskripsi tidak tersedia." } } fun ExerciseInfo.getMainImageUrl(): String? { return images?.firstOrNull { it.isMain }?.imageUrl ?: images?.firstOrNull()?.imageUrl } fun ExerciseInfo.getMainVideoUrl(): String? { return videos?.firstOrNull { it.isMain }?.videoUrl ?: videos?.firstOrNull()?.videoUrl } </pre>
--	---

Table 23. Source Code Jawaban Soal 1

9. ExerciseInfoEntity

1	<code>package com.example.zennfit.data</code>
2	<code>import androidx.room.Entity</code>
3	<code>import androidx.room.PrimaryKey</code>
4	<code>@Entity(tableName = "exercise_info_table")</code>
5	<code>data class ExerciseInfoEntity(</code>
6	<code> @PrimaryKey val id: Int,</code>
7	<code> val name: String,</code>
8	<code> val description: String,</code>
	<code> val imageUrl: String?,</code>
	<code> val videoUrl: String?</code>
	<code>)</code>

Table 24. Source Code Jawaban Soal 1

10. ExerciseInfoDao

1	<code>package com.example.zennfit.data</code>
2	<code>import androidx.room.Dao</code>
3	<code>import androidx.room.Insert</code>
4	<code>import androidx.room.OnConflictStrategy</code>
5	<code>import androidx.room.Query</code>
6	<code>import kotlinx.coroutines.flow.Flow</code>
	<code>@Dao</code>
	<code>interface ExerciseInfoDao {</code>

7	<code>@Query("SELECT * FROM exercise_info_table ORDER BY name ASC")</code>
8	<code>fun getAllExerciseInfo(): Flow<List<ExerciseInfoEntity>></code>
9	<code>@Insert(onConflict = OnConflictStrategy.REPLACE)</code>
10	<code>suspend fun insertAll(exerciseInfo: List<ExerciseInfoEntity>)</code>
11	<code>@Query("DELETE FROM exercise_info_table")</code>
	<code>suspend fun deleteAllExerciseInfo()</code>
	<code>}</code>

Table 25. Source Code Jawaban Soal 1

11. AppDataBase

1	<code>package com.example.zennfit.data</code>
2	<code>import android.content.Context</code>
3	<code>import androidx.room.Database</code>
4	<code>import androidx.room.Room</code>
5	<code>import androidx.room.RoomDatabase</code>
6	<code>@Database(entities = [ExerciseInfoEntity::class], version = 1,</code>
7	<code>exportSchema = false)</code>
8	<code>abstract class AppDatabase : RoomDatabase() {</code>
9	<code> abstract fun exerciseInfoDao(): ExerciseInfoDao</code>
10	<code> companion object {</code>
11	<code> @Volatile</code>
	<code> private var INSTANCE: AppDatabase? = null</code>
	<code> fun getDatabase(context: Context): AppDatabase {</code>
	<code> return INSTANCE ?: synchronized(this) {</code>
	<code> val instance = Room.databaseBuilder(</code>
	<code> context.applicationContext,</code>
	<code> AppDatabase::class.java,</code>
	<code> "zennfit_database"</code>
	<code>).build()</code>
	<code> INSTANCE = instance</code>
	<code> instance</code>
	<code> }</code>
	<code> }</code>
	<code> }</code>
	<code>}</code>

Table 26. Source Code Jawaban Soal 1

12. AlatGymRepository

1	<code>package com.example.zennfit.data</code>
2	<code>import android.util.Log</code>
3	<code>import com.example.zennfit.model.*</code>
4	<code>import com.example.zennfit.network.WgerApiService</code>
5	<code>import kotlinx.coroutines.Dispatchers</code>


```

4 import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flowOn
5 import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.onEach
6 import kotlinx.coroutines.flow.catch
import kotlinx.coroutines.coroutineScope
7 import kotlinx.coroutines.withContext
8
9 class AlatGymRepository(
10     private val wgerApiService: WgerApiService,
11     private val exerciseInfoDao: ExerciseInfoDao
12 ) {
13     fun getExerciseInfo(searchQuery: String? = null):
Flow<List<ExerciseInfo>> = exerciseInfoDao.getAllExerciseInfo()
14         .map { entities ->
15             Log.d("AlatGymRepositoryFlow", "Mapping ${entities.size}
items from Room entities to external models.")
16             entities.map { it.toExternalModel() }
17         }
18         .onEach {
19             Log.d("AlatGymRepositoryFlow", "Flow emitted data to
ViewModel (from Room): ${it.size} items.")
20
21             if (it.isEmpty() || searchQuery != null) {
22                 Log.d("AlatGymRepositoryFlow", "Attempting to fetch
from network with query: '${searchQuery ?: "none"}'")
23                 fetchAndSaveToRoom(searchQuery)
24             } else {
25                 Log.d("AlatGymRepositoryFlow", "Room contains data,
skipping immediate network fetch for now.")
26             }
27         }
28         .catch { e ->
29             Log.e("AlatGymRepositoryFlow", "Error in flow collection:
${e.message}", e)
30             emit(emptyList())
31         }
32         .flowOn(Dispatchers.IO)
33
34     private suspend fun fetchAndSaveToRoom(query: String?) {
35         coroutineScope {
36             try {
37                 Log.d("AlatGymRepositoryNetwork", "Starting network
fetch with query: '${query ?: "none"}'")
38                 val response = wgerApiService.getExerciseInfo(query)
39                 Log.d("AlatGymRepositoryNetwork", "Network response
received. isSuccessful: ${response.isSuccessful}, Code:
${response.code()}, Message: ${response.message()}")
40
41                 if (response.isSuccessful) {
42                     val apiResponse = response.body()
43                     if (apiResponse != null) {
44                         val exerciseInfoList = apiResponse.results
45                         if (exerciseInfoList != null &&
exerciseInfoList.isNotEmpty()) {
46                             Log.d("AlatGymRepositoryNetwork", "API
response body is not null. Results size: ${exerciseInfoList.size}")

```

```

        val exerciseInfoEntities =
exerciseInfoList.map { it.toEntity() }

        exerciseInfoDao.deleteAllExerciseInfo()

exerciseInfoDao.insertAll(exerciseInfoEntities)
        Log.d("AlatGymRepositoryNetwork",
"Successfully fetched ${exerciseInfoList.size} items from network and
saved to Room.")

    } else {
        Log.w("AlatGymRepositoryNetwork", "API
response results are null or empty for query: '${query ?: "none"}'")
        if (query != null) {

exerciseInfoDao.deleteAllExerciseInfo()
            Log.d("AlatGymRepositoryNetwork",
"Cleared Room data as network search for '${query}' returned empty.")
        }
    }
    } else {
        Log.w("AlatGymRepositoryNetwork", "API
response body is null.")
    }
    } else {
        Log.e("AlatGymRepositoryNetwork", "API call
failed with code: ${response.code()}, message: ${response.message()},
Raw Error Body: ${response.errorBody()?.string()}")
    }
    } catch (e: Exception) {
        Log.e("AlatGymRepositoryNetwork", "Error during
network fetch for query '${query ?: "none"}': ${e.message}", e)
    }
}

suspend fun getExerciseInfoById(id: Int): ExerciseInfo? {
    return withContext(Dispatchers.IO) {
        try {
            Log.d("AlatGymRepositoryById", "Attempting to fetch
exercise with ID: $id")
            val response = wgerApiService.getExerciseInfoById(id)
            Log.d("AlatGymRepositoryById", "Response for ID $id:
isSuccessful: ${response.isSuccessful}, Code: ${response.code()},
Message: ${response.message()}")

            if (response.isSuccessful) {
                val exercise = response.body()
                if (exercise != null) {
                    Log.d("AlatGymRepositoryById", "Successfully
fetched exercise ID $id: ${exercise.localizedName}")
                    exercise
                } else {
                    Log.w("AlatGymRepositoryById", "API response
body for ID $id is null.")
                    null

```

```

    }
    } else {
        Log.e("AlatGymRepositoryById", "API call failed
for ID $id with code: ${response.code()}, message:
${response.message()}, Raw Error Body:
${response.errorBody()?.string()}")
        null
    }
    } catch (e: Exception) {
        Log.e("AlatGymRepositoryById", "Error fetching
exercise with ID $id: ${e.message}", e)
        null
    }
}

}

private fun ExerciseInfo.toEntity(): ExerciseInfoEntity {
    return ExerciseInfoEntity(
        id = this.id,
        name = this.localizedName(),
        description = this.localizedDescription(),
        imageUrl = this.mainImageUrl(),
        videoUrl = this.mainVideoUrl()
    )
}

private fun ExerciseInfoEntity.toExternalModel(): ExerciseInfo {
    return ExerciseInfo(
        id = this.id,
        uuid = "",
        created = null,
        lastUpdate = null,
        lastUpdateGlobal = null,
        category = null,
        muscles = emptyList(),
        musclesSecondary = emptyList(),
        equipment = emptyList(),
        license = null,
        licenseAuthor = null,
        images = if (this.imageUrl != null)
listOf(ExerciseImage(0, null, this.imageUrl, true, "")) else null,
        videos = if (this.videoUrl != null)
listOf(ExerciseVideo(0, null, this.videoUrl, true, "")) else null,
        translations = listOf(
            ExerciseTranslation(
                id = 0,
                uuid = "",
                name = this.name,
                exerciseId = this.id,
                description = this.description,
                created = null,
                languageId = 2,
                aliases = emptyList(),
                notes = emptyList(),
                license = null,
                licenseTitle = null,
                licenseObjectUrl = null,

```

	<pre> licenseAuthor = null, licenseAuthorUrl = null, licenseDerivativeSourceUrl = null, authorHistory = emptyList()), variations = null, authorHistory = emptyList(), totalAuthorsHistory = emptyList()) } } </pre>
--	---

Table 27. Source Code Jawaban Soal 1

13. AlatGymAdapter

1	<code>package com.example.zennfit</code>
2	<code>import android.view.LayoutInflater</code>
3	<code>import android.view.ViewGroup</code>
4	<code>import android.content.Intent</code>
5	<code>import android.net.Uri</code>
6	<code>import android.view.View</code>
7	<code>import androidx.recyclerview.widget.RecyclerView</code>
8	<code>import com.example.zennfit.databinding.ListItemBinding</code>
9	<code>import com.example.zennfit.model.ExerciseInfo</code>
10	<code>import com.bumptech.glide.Glide</code>
11	<code>import com.example.zennfit.model.getLocalizedName</code>
	<code>import com.example.zennfit.model.getLocalizedDescription</code>
	<code>import com.example.zennfit.model.getMainImageUrl</code>
	<code>import android.util.Log</code>
	<code>class AlatGymAdapter(</code>
	<code> private var alatList: List<ExerciseInfo>,</code>
	<code> private val onClick: (ExerciseInfo) -> Unit</code>
	<code>) : RecyclerView.Adapter<AlatGymAdapter.ViewHolder>() {</code>
	<code> class ViewHolder(val binding: ListItemBinding) :</code>
	<code>RecyclerView.ViewHolder(binding.root)</code>
	<code> override fun onCreateViewHolder(parent: ViewGroup, viewType:</code>
	<code>Int): ViewHolder {</code>
	<code> val binding =</code>
	<code>ListItemBinding.inflate(LayoutInflater.from(parent.context), parent,</code>
	<code>false)</code>
	<code> return ViewHolder(binding)</code>
	<code> }</code>
	<code> override fun onBindViewHolder(holder: ViewHolder, position: Int)</code>
	<code>{</code>
	<code> val alat = alatList[position]</code>
	<code> holder.binding.namaAlat.text = alat.getLocalizedName()</code>
	<code> val description = alat.getLocalizedDescription()</code>

```

        if (description == "Deskripsi tidak tersedia." ||
description.isBlank()) {
            holder.binding.descDetail.visibility = View.GONE
        } else {
            holder.binding.descDetail.visibility = View.VISIBLE
            holder.binding.descDetail.text = description
        }

        val imageUrl = alat.getMainImageUrl()
        if (!imageUrl.isNullOrEmpty()) {
            holder.binding.alatImageList.visibility = View.VISIBLE
            Glide.with(holder.itemView.context)
                .load(imageUrl)
                .placeholder(R.drawable.placeholder_image)
                .error(R.drawable.placeholder_image)
                .into(holder.binding.alatImageList)
        } else {
            holder.binding.alatImageList.visibility = View.GONE
        }

        holder.binding.viewButton.setOnClickListener {
            onClick(alat)
        }

        holder.binding.browserButton.setOnClickListener {
            val context = holder.itemView.context
            val wgerUrl =
"https://wger.de/id/exercise/${alat.id}/view"

            Log.d("BrowserUrlDebug", "Final URL sent to browser:
$wgerUrl")

            val intent = Intent(
                Intent.ACTION_VIEW,
                Uri.parse(wgerUrl)
            )

            try {
                context.startActivity(intent)
                Log.d("BrowserUrlDebug", "URL opened successfully.")
            } catch (e: Exception) {
                Log.e("BrowserUrlDebug", "Could not open URL:
$wgerUrl. Error: ${e.message}")
            }
        }

        override fun getItemCount(): Int {
            val count = alatList.size
            Log.d("AlatGymAdapter", "getItemCount() called. Returning:
$count")
            return count
        }

        fun updateData(newList: List<ExerciseInfo>) {
            Log.d("AlatGymAdapter", "updateData() called with
${newList.size} items.")

```

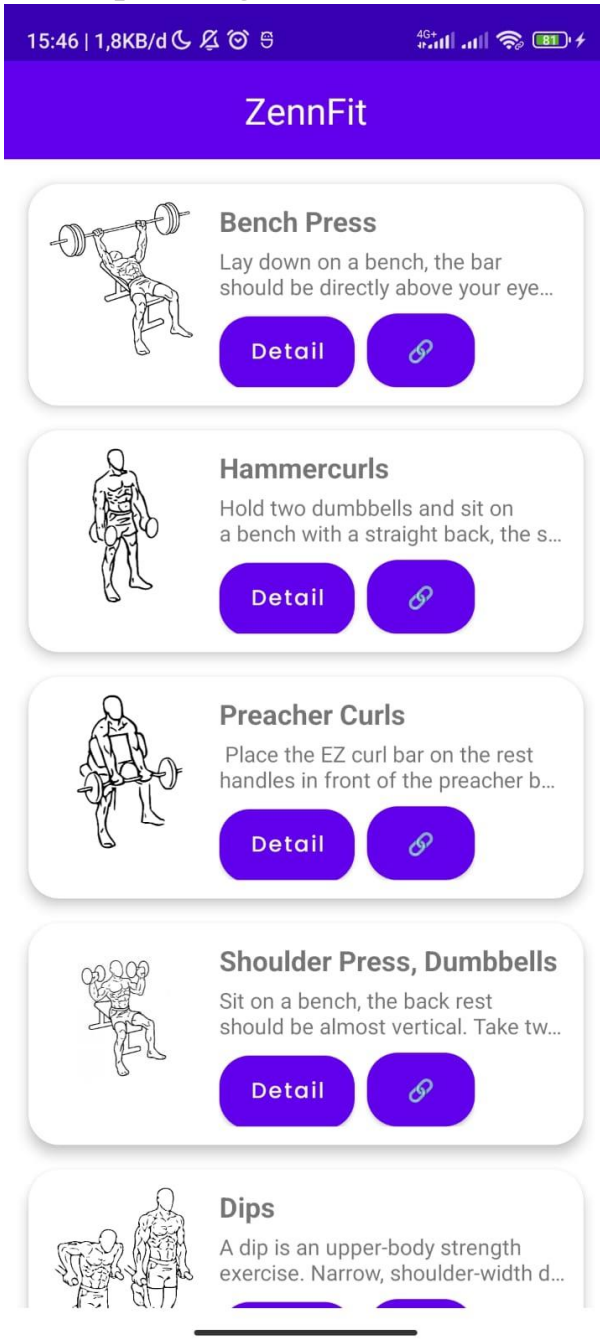
```

    alatList = newList
    notifyDataSetChanged()
    Log.d("AlatGymAdapter", "notifyDataSetChanged() called.")
}
}

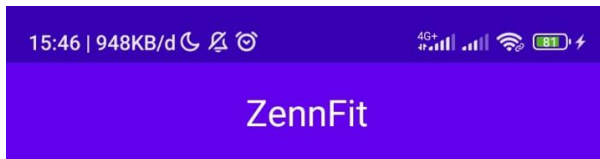
```

Table 28. Source Code Jawaban Soal 1

B. Output Program



Gambar 8. Screenshot hasil soal 1



Bench Press

Lay down on a bench, the bar should be directly above your eyes, the knees are somewhat angled and the feet are firmly on the floor. Concentrate, breath deeply and grab the bar more than shoulder wide. Bring it slowly down till it briefly touches your chest at the height of your nipples. Push the bar up.

If you train with a high weight it is advisable to have a spotter that can help you up if you can't lift the weight on your own.

With the width of the grip you can also control which part of the chest is trained more:

wide grip: outer chest muscles

narrow grip: inner chest muscles and triceps



Gambar 9. Screenshot hasil soal 1

C. Pembahasan

1. HomeFragment

Langsung saja ke `onCreateView()` pada kode `_binding = FragmentHomeBinding.inflate(inflater, container, false)` ini melakukan inflasi

(mengubah XML menjadi View object), kemudian menyimpan View tersebut ke `_binding`. Lalu `onViewCreated`, isinya menginisialisasi `RecyclerView.Adapter` untuk daftar alat gym. Digunakan ketika user klik salah satu item maka akan memicu `viewModel.onItemClicked()` dan mencetak nama alat yang diklik ke Log. Pada `recyclerView` setup itu mengatur `RecyclerView` agar menampilkan daftar alat secara vertikal. lalu kode ini `viewModel.alatList.collectLatest { ... }` mengamati perubahan pada data list alat dari `ViewModel` dan memperbarui adapter ketika ada data baru. `viewModel.selectedItem.collectLatest { selected -> ... }` ini untuk navigasi ke bagian detail, setelah navigasi, item yang dipilih akan di-clear dari `ViewModel` dengan `viewModel.clearSelectedItem()`. Terakhir `_binding = null` ini membersihkan `_binding` untuk menghindari memory leaks saat view dihancurkan (misalnya saat berpindah fragment).

2. DetailFragment

`class DetailFragment : Fragment()` { ini adalah subclass dari `Fragment` yang digunakan untuk menampilkan detail dari alat gym terpilih, termasuk nama, deskripsi, gambar, dan video. pada View Binding, Menggunakan View Binding (`FragmentDetailBinding`) untuk mengakses elemen XML `fragment_detail.xml`. `_binding` digunakan sebagai penampung sementara; binding digunakan untuk akses aman. pada bagian `ExoPlayer`, ini berisi player: objek `ExoPlayer` untuk memutar video. `playWhenReady`: status apakah video langsung diputar setelah `prepare()`. `playbackPosition`: menyimpan posisi terakhir saat video dijeda atau dihentikan. Selanjutnya fungsi `newInstance`, yaitu membuat fragment dengan membawa data (nama, deskripsi, URL gambar, dan URL video) sebagai argumen `Bundle` dan digunakan saat berpindah dari `HomeFragment`. Lalu `onCreateView` dimana ini meng-inflate layout XML `fragment_detail.xml` menggunakan ViewBinding. Pada bagian `onViewCreated` mengambil argumen lalu dikirim melalui `newInstance` lalu menampilkan data tadi ke UI dengan binding. Selanjutnya menampilkan gambar dengan kondisional, Jika `imageUrl` ada, tampilkan gambar menggunakan library Glide. Bila tidak ada gambar, sembunyikan `ImageView`. Lalu menampilkan video juga jika `videoUrl` tersedia, tampilkan `PlayerView` dan inisialisasi `ExoPlayer`. Lalu menginisialisasi `ExoPlayer` yaitu membuat instance `ExoPlayer` dan mempersiapkan media dari URL video lalu atur video agar bisa autoplay (`playWhenReady`) dan mengulang terus (`REPEAT_MODE_ALL`). lalu ke `onStart` dimana jika player belum ada dan URL video tersedia, player diinisialisasi ulang. `onResume` ini melanjutkan playback saat fragment aktif kembali. `onPause` ini simpan posisi video dan status saat fragment tidak aktif. `onStop` dan `releasePlayer` bagian ini akan melepaskan semua resource player saat fragment tidak lagi tampil, mencegah memory leak. Terakhir `onDestroyView` membersihkan binding untuk mencegah memory leak.

3. AlatGymViewModel

Pertama deklarasi viewModel, repository: dependensi untuk mengambil data dari sumber (bisa API/local DB). lalu masuk ke state list dan item terpilih yang isinya `_alatList`: state privat yang menyimpan list alat gym. `alatList`: versi publik (immutable) untuk di-observe dari UI. `_selectedItem`: item yang sedang dipilih. `selectedItem`: expose agar hanya bisa dibaca. Lalu ada daftar ID, ID ini adalah ID dari latihan tertentu yang ingin ditampilkan saja (terkurasi). Mewakili latihan penting seperti Bench Press, Hammer Curls, dll. Lalu ini blok dipanggil saat ViewModel pertama kali dibuat lalu memanggil fungsi `loadCuratedExercises()` untuk mulai load data. Masuk ke fungsi `loadCuratedExercises`, Coroutine diluncurkan menggunakan `viewModelScope.launch` (untuk operasi async) lalu untuk setiap id dalam `curatedExerciseIds`, ambil data dari repository, cek apakah data memiliki gambar, deskripsi, dan video. Jika data lengkap, tambahkan ke list `loadedExercises`, jika tidak, lewati. Setelah semua ID dicek, hasil akhir disimpan ke `_alatList`. Selanjutnya fungsi `onItemClicked` yang menyimpan item yang dipilih pengguna ke `selectedItem` dan untuk menampilkan detail item di UI. Fungsi `clearSelectedItem` ini menghapus pilihan pengguna dan dipanggil saat pengguna kembali ke list atau membatalkan aksi.

4. AlatGymViewModelFactory

Pertama mendefinisikan `AlatGymViewModelFactory` yang mengimplementasi `ViewModelProvider.Factory`. repository disimpan sebagai properti karena nanti akan diberikan ke ViewModel. override fun `<T : ViewModel> create(modelClass: Class<T>): T` { method ini dipanggil waktu `ViewModelProvider` ingin membuat ViewModel. Tipe T adalah tipe ViewModel yang ingin dibuat. Lalu masuk ke kondisional, dicek dulu apakah class yang diminta adalah `AlatGymViewModel`. kalau ya, buat objek `AlatGymViewModel` dan kasih parameter repository. as T digunakan untuk meng-cast ke tipe generic T, dan `@Suppress("UNCHECKED_CAST")` untuk menenangkan warning. pada kode ini throw `IllegalArgumentException("Unknown ViewModel class")` jika ternyata class yang diminta bukan `AlatGymViewModel`, maka dilempar error karena kita nggak tahu cara buat ViewModel lain.

5. AlatGymAdapter

class `AlatGymAdapter` di dalamnya ada `alatList` ini untuk daftar data alat gym bertipe `ExerciseInfo` yang akan ditampilkan. `onClick` adalah Lambda function yang dijalankan ketika tombol "view" diklik pada item. class `ViewHolder(val binding: ListItemBinding) : RecyclerView.ViewHolder(binding.root)` viewHolder menyimpan binding dari `list_item.xml`, agar efisien dalam memetakan data ke view. Lalu di `onCreateViewHolder` membuat instance `ViewHolder` dengan memanfaatkan `ViewBinding` untuk meng-inflate `list_item.xml`. `onBindViewHolder`, override fun

onBindViewHolder(holder: ViewHolder, position: Int) ini menampilkan data ExerciseInfo ke tampilan yaitu ada nama, deskripsi, gambar, Tombol view yang ketika tombol "View" diklik, akan men-trigger onClick lambda dari konstruktor, tombol browser yaitu membuat URL untuk membuka halaman detail alat di website wger.de. dan Intent ACTION_VIEW digunakan untuk membuka URL di browser default, jika intent berhasil dibuka, log sukses dan jika gagal (tidak ada browser, dll.), log error. Lalu ada getItemCount yang akan mengembalikan jumlah item yang akan ditampilkan di RecyclerView dan log jumlah data untuk debugging. updateData ini method publik untuk memperbarui data alatList dan memanggil notifyDataSetChanged() agar RecyclerView me-refresh tampilannya.

6. WgerApiService

Langsung ke interface WgerApiService, ini berisi deklarasi fungsi-fungsi untuk mengambil data dari API Wger (Workout Manager Exercise API). Nantinya, Retrofit akan meng-generate implementasi dari interface ini secara otomatis. Selanjutnya Mengambil daftar latihan berdasarkan kata kunci pencarian (query). @GET("api/v2/exerciseinfo/"): melakukan HTTP GET ke endpoint tersebut. @Query("search"): menambahkan parameter pencarian di URL. suspend: karena ini fungsi suspend, maka bisa dipanggil dalam coroutine (viewModelScope.launch). Mengembalikan Response<WgerExerciseInfoResponse>, yaitu respon yang isinya daftar latihan. Selanjutnya Mengambil satu data latihan berdasarkan id. @GET("api/v2/exerciseinfo/{id}/"): endpoint dengan parameter path ({id}). @Path("id"): mengganti {id} di URL dengan nilai id yang diberikan. Return: Response<ExerciseInfo> yang berisi satu objek latihan (ExerciseInfo).

7. ApiClient

private const val BASE_URL = "https://wger.de/" ini adalah URL dasar untuk semua endpoint API. Lalu bagian Logging Interceptor berisi HttpLoggingInterceptor dari OkHttp digunakan untuk melihat log permintaan dan respons HTTP. Level.BODY berarti seluruh isi dari request dan response body akan dicetak di Logcat (penting saat debugging API). OkHttpClient, HTTP client yang digunakan oleh Retrofit untuk menjalankan request. Di sini menambahkan interceptor agar semua aktivitas HTTP bisa dilihat melalui log. Menetapkan batas waktu (timeout) untuk koneksi, membaca, dan menulis selama 30 detik. Selanjutnya ke bagian Retrofit Builder , Retrofit.Builder() digunakan untuk membuat instance Retrofit. baseUrl(BASE_URL): menetapkan URL dasar untuk API. client(okHttpClient): Retrofit menggunakan client dengan konfigurasi timeout dan logging tadi. addConverterFactory(GsonConverterFactory.create()): gunakan Gson untuk mengubah data JSON menjadi objek Kotlin (ExerciseInfo, WgerExerciseInfoResponse, dll). by lazy artinya instance Retrofit hanya akan dibuat satu kali saat pertama kali dipanggil. Terakhir akan menghasilkan service API, membuat instance dari WgerApiService, interface yang didefinisikan untuk endpoint API. retrofit.create(...) akan meng-generate implementasi untuk interface tersebut.

Menggunakan by lazy, objek wgerApiService juga hanya akan dibuat saat pertama kali digunakan.

8. WgerEquipmentInfoResponse

Data class WgerExerciseInfoResponse ini adalah data class yang digunakan untuk menyimpan data. Otomatis menyediakan toString(), equals(), hashCode(), dan copy(). @SerializedName("count") val count: Int, mewakili jumlah total data exercise yang tersedia. @SerializedName("next") val next: String?, URL halaman berikutnya (untuk pagination) dan bertipe nullable (String?) karena bisa null jika tidak ada halaman selanjutnya. @SerializedName("previous") val previous: String?, URL halaman sebelumnya (juga untuk pagination) dan bertipe nullable. @SerializedName("results") val results: List<ExerciseInfo> ini adalah list dari data latihan (exercise) yang dikembalikan pada halaman tersebut. Tipe datanya adalah List<ExerciseInfo>, artinya respons ini akan berisi banyak objek ExerciseInfo (data class lain yang mewakili satu item exercise).

9. ExerciseInfo

Data Class Utama: ExerciseInfo ini model utama yang mewakili satu data latihan/exercise. Field-nya adalah hasil parsing langsung dari API JSON. Data class Category(val id: Int, val name: String) kategori latihan, seperti: Chest, Back, Legs. data class Muscle ini menjelaskan otot yang dilatih: isFront: apakah otot ada di bagian depan tubuh. imageUrlMain dan imageUrlSecondary: link gambar otot. data class EquipmentDetail(val id: Int, val name: String) nama alat yang digunakan. data class License ini informasi lisensi konten seperti: Nama lengkap lisensi (Creative Commons), URL lisensi. data class ExerciseImage ini adalah gambar-gambar yang berkaitan dengan latihan, termasuk yang ditandai sebagai gambar utama (isMain). data class ExerciseVideo ini video latihan (jika tersedia), juga bisa punya status utama (isMain). data class ExerciseTranslation(...) ini berisi informasi terjemahan seperti nama dan deskripsi latihan dalam berbagai bahasa, ID bahasa (languageId = 2 untuk Bahasa Indonesia), deskripsi bisa berupa HTML, termasuk info lisensi terjemahan. Pada kode ini fun ExerciseInfo.getLocalizedString(languageId: Int = 2): String akan mengambil nama latihan berdasarkan bahasa tertentu (default: 2 = Bahasa Indonesia). Jika tidak tersedia, ambil yang pertama. Jika tetap tidak ada, tampilkan "Nama tidak tersedia". Kode ini fun ExerciseInfo.getLocalizedDescription(languageId: Int = 2): String ini sama seperti di atas, tapi untuk deskripsi latihan. Deskripsi biasanya dalam format HTML, jadi diformat ke plain text dengan Html.fromHtml. Menyesuaikan versi Android agar kompatibel (Build.VERSION.SDK_INT). Output: teks deskripsi latihan dalam bahasa lokal. fun ExerciseInfo.getMainImageUrl(): String? ini mengambil URL gambar utama. Jika tidak ada gambar utama (isMain == true), ambil gambar pertama jika ada. fun ExerciseInfo.getMainVideoUrl(): String? ini juga sama seperti

gambar, tetapi untuk video. Mengambil video utama, atau fallback ke video pertama jika isMain tidak ada.

10. ExerciseInfoEntity

Pertama deklarasi Entity ExerciseInfoEntity, `@Entity(tableName = "exercise_info_table")` ini memberitahu Room bahwa class ExerciseInfoEntity ini akan menjadi tabel dalam SQLite database. `tableName = "exercise_info_table"` adalah nama tabel yang akan digunakan di database. Ini akan dibuat oleh Room secara otomatis saat database dibuat. data class ExerciseInfoEntity ini adalah data class Kotlin yang menyimpan data sederhana dan digunakan untuk mewakili satu baris dari tabel exercise_info_table.

11. ExerciseInfoDao

interface ExerciseInfoDao { ini komponen penting dalam arsitektur Room, bertindak sebagai jembatan antara database dan layer aplikasi lainnya (biasanya repository dan ViewModel). Lalu mengambil semua data dengan `FROM exercise_info_table ORDER BY name ASC` mengambil semua data dari tabel exercise_info_table, lalu mengurutkannya berdasarkan nama (name) secara ascending (A → Z). `Flow<List<ExerciseInfoEntity>>`: menggunakan Kotlin Flow agar data bisa berubah secara real-time dan langsung merefleksikan perubahan ke UI. Fungsi ini bersifat non-suspend karena Flow itu sendiri bersifat asynchronous dan dieksekusi secara coroutine-aware. lalu masukkan data dengan `@Insert(onConflict = OnConflictStrategy.REPLACE)` suspend fun `insertAll(exerciseInfo: List<ExerciseInfoEntity>)`. `@Insert` ini memberitahu Room bahwa fungsi ini untuk menambahkan data ke tabel. `onConflict = OnConflictStrategy.REPLACE` jika ada data dengan id yang sama (primary key), maka data lama akan diganti dengan data baru. `suspend`: Fungsi ini adalah suspend function, yang artinya harus dipanggil dari coroutine atau fungsi suspend lainnya. Ini agar tidak memblokir UI thread. `exerciseInfo: List<ExerciseInfoEntity>` fungsi ini menerima daftar (list) entitas latihan untuk dimasukkan sekaligus. Menghapus semua data dengan `@Query("DELETE FROM exercise_info_table")` suspend fun `deleteAllExerciseInfo()`. `DELETE FROM exercise_info_table` ini menghapus semua data dari tabel exercise_info_table. `suspend`: Karena proses penghapusan bisa memakan waktu (tergantung ukuran data), fungsi ini harus dijalankan di coroutine agar tidak membebani UI thread.

12. AppDataBase

Pertama ada anotasi database, entities: Menentukan tabel-tabel yang digunakan dalam database ini. Di sini hanya ada 1 entitas yaitu ExerciseInfoEntity. `version = 1`: Menentukan versi database. Versi ini penting untuk migrasi database jika struktur berubah di masa depan. `exportSchema = false`: Jika true, Room akan menyimpan file skema (struktur database) untuk dokumentasi atau pengujian. Di sini dimatikan. `abstract class AppDatabase : RoomDatabase()` { ini adalah RoomDatabase yang

digunakan Room untuk membuat dan mengelola database. abstract dimana kelas ini tidak bisa diinstansiasi langsung, tapi akan dibuat oleh Room saat runtime. abstract fun exerciseInfoDao(): ExerciseInfoDao fungsi abstrak ini memberi akses ke DAO (ExerciseInfoDao), yang menangani query terhadap tabel exercise_info_table. Lalu ada singleton Companion Object, companion object ini digunakan agar fungsi dan properti di dalamnya bersifat statis (seperti static class di Java). INSTANCE untuk menyimpan satu-satunya instance tunggal (singleton) dari AppDatabase. Tujuannya agar database tidak dibuat berulang kali. @Volatile ini untuk memastikan bahwa perubahan nilai INSTANCE dapat terlihat antar thread, penting untuk thread-safety. Lalu Fungsi getDatabase(context: Context), INSTANCE ?: synchronized(this) ini jika INSTANCE belum ada, buat satu dalam blok sinkronisasi agar aman dari kondisi balapan antar-thread. Room.databaseBuilder(...) Room membuat database dengan context.applicationContext untuk menghindari memory leak dari context, AppDatabase::class.java ini kelas database yang ingin dibangun, zennfit_database ini nama file database SQLite yang akan disimpan di memori perangkat. INSTANCE = instance ini untuk menyimpan instance ke dalam variabel static agar tidak dibuat ulang.

13. AlatGymRepository

Didalam class AlatGymRepository ada wgerApiService yaitu objek Retrofit yang digunakan untuk mengambil data dari API dan exerciseInfoDao ini DAO dari Room Database untuk operasi lokal seperti insert, delete, dan get data. fun getExerciseInfo(searchQuery: String? = null): Flow<List<ExerciseInfo>> fungsi ini mengambil data dari database lokal (Room) terlebih dahulu dan hanya mengakses API jika diperlukan. Menggunakan Flow dari Kotlin Coroutines agar data bisa diamati secara real-time dan asynchronous. Fungsi fetchAndSaveToRoom(), private suspend fun fetchAndSaveToRoom(query: String?) fungsi ini mengambil data dari API dan menyimpannya ke database lokal Room. Fungsi getExerciseInfoById(), suspend fun getExerciseInfoById(id: Int): ExerciseInfo? fungsi ini digunakan untuk mengambil satu data latihan berdasarkan ID langsung dari API, bukan dari Room. Lalu dalam Fungsi Konversi Data, toEntity() ini mengubah model dari API (ExerciseInfo) ke bentuk database (ExerciseInfoEntity) mengambil nama, deskripsi, image URL, dan video URL dari model API. toExternalModel() ini untuk mengubah ExerciseInfoEntity dari Room ke model untuk ditampilkan di UI (private fun ExerciseInfoEntity.toExternalModel(): ExerciseInfo), mengisi properti ExerciseInfo dari Room. Karena database hanya menyimpan name, description, imageUrl, dan videoUrl, maka properti lainnya (muscles, equipment, dll) diset null atau emptyList().

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Specter735/Pemrograman-Mobile.git>